



ECOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES

www.esprit.tn - E-mail : contact@esprit.tn

Siège Social : 18 rue de l'Usine - Charguia II - 2035 - Tél. : +216 71 941 541 - Fax. : +216 71 941 889

Annexe : Z.I. Chotrana II - B.P. 160 - 2083 - Pôle Technologique - El Ghazala - Tél. : +216 70 685 685 - Fax. : +216 70 685 454

2023 - 2024

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : INFORMATIQUE

**Conception et développement d'une
application de gestion de ressources
« ARYANEGO »**

Réalisé par : Mohamed DHAHBI

Encadrant ESPRIT : Mme. Lilia GOSSA

Encadrant Entreprise : M. Ahmed Ben OUZDEOU



Je valide le dépôt du rapport PFE relatif à l'étudiant nommé ci-dessous / I
validate the submission of the student's report:

- Nom & Prénom /Name & Surname : Dhahbi Mohamed

Encadrant Entreprise/ Business site Supervisor

- Nom & Prénom /Name & Surname : Belhaj Abdallah Maher

Cachet & Signature / Stamp & Signature



Encadrant Académique/Academic Supervisor

- Nom & Prénom /Name & Surname : Gossa Lilia

Signature / Signature

A handwritten signature in black ink, appearing to read 'Lilia Gossa'.

Ce formulaire doit être rempli, signé et scanné/This form must be completed, signed and scanned. Ce formulaire doit être introduit après la page de garde/ This form must be inserted after the cover page.

INTRODUCTION GENERALE

L'industrie télécoms est en pleine mutation, avec une augmentation significative de la demande de services, une augmentation des investissements et une accélération du déploiement des réseaux mobiles. Cette expansion fulgurante pose de nouveaux défis, en particulier en ce qui concerne le développement durable et les infrastructures. Depuis dix ans, ce secteur s'est imposé comme un centre économique mondial, porté par l'émergence d'Internet et la création de nouveaux marchés à travers le globe. En 2021, le marché des télécommunications à l'échelle mondiale était déjà composé de 4,3 millions de pylônes, avec une croissance annuelle de 3,34% et nous avons pour objectif et de soutenir la transition vers le numérique pour obtenir un outil des gestions amélioré.

C'est dans ce cadre que se place notre projet de fin d'étude. En effet, il nous a été confié de développer une application web sous le nommée « ARYANEGO ». Notre application représente un moyen simplifiant la mise en relation entre les clients et les bailleurs qui souhaitent louer ou vendre des pylônes.

Le présent rapport présente les différentes étapes de la réalisation et l'automatisation de notre projet et s'étalera sur 5 chapitres :

- Le premier chapitre intitulé « **Contexte général du projet** » présente le contexte général du projet et les solutions existantes ainsi que la solution proposée. La méthodologie de travail est également discutée dans ce chapitre.

- Le deuxième chapitre intitulé « Analyse et spécification des besoins » est consacré à la spécification des besoins fonctionnels et non fonctionnels, ainsi que le pilotage du projet par la méthode Scrum

- Les trois chapitres suivants seront dédiés à la mise en œuvre des trois releases de notre application web, où nous nous intéressons à l'élaboration des sprints. Chaque sprint sera réparti sur trois parties : analyse, conception et réalisation.

CHAPITRE 1

CONTEXTE GÉNÉRAL DU PROJET

Introduction

Dans cette introduction, nous débuterons par décrire le contexte général de notre projet. Ensuite, nous détaillerons l'idée générale du projet. Nous identifierons les défis que nous devons relever avant d'examiner les approches actuelles, en les évaluant rigoureusement et en exposant notre solution recommandée. Pour clôturer cette section, nous procéderons à une analyse approfondie des diverses méthodologies de développement, en opérant un comparatif critique pour déterminer notre méthode optimale.

1. Contexte générale du projet

Cette partie présente le cadre général du projet ainsi que la présentation de l'entreprise.

1.1 Cadre du projet

Ce projet constitue une partie intégrante de mon projet de fin d'études pour l'obtention du diplôme national d'ingénieur en informatique, délivré par l'École Supérieure Privée d'Ingénierie et de Technologies (ESPRIT). Mon stage, d'une durée de six mois, s'effectue au sein de la société Aryason Consulting.

1.2 Présentation de l'organisme d'accueil

1.2.1 Présentation de la société

Aryason Consulting est un cabinet de consulting IT technologies de l'information spécialisé dans les nouvelles technologies (Cloud, Big Data, IA) qui accompagne les entreprises dans leur transformation numérique [1]. Créé en 2013 par des jeunes entrepreneurs, qui ont réussi à développer des outils de gestion d'approvisionnement en ligne. La société Aryason Consulting vise à faciliter le flux des activités quotidiennes des clients, en leur proposant des outils informatiques qui les aideront à améliorer leur travail.

✓ Fiche signalétique :

- Raison Sociale : Aryason Consulting
- Date de création : 2013
- Adresse : Villejuif, Paris
- Téléphone : +(33) 6 22 28 7293
- Email : contact@aryason.fr
- Site web: <http://www.aryason.fr/>

✓ Services :

- Branding
- Conception graphique
- Développement des systèmes de gestion
- Personnalisation des services
- Développement Web

✓ Valeurs :

- Expertise
- Transparence
- Partage

1.2.2 Organisation

La **Figure 1** présente l'équipe administrative de l'entreprise « Aryason Consulting ».

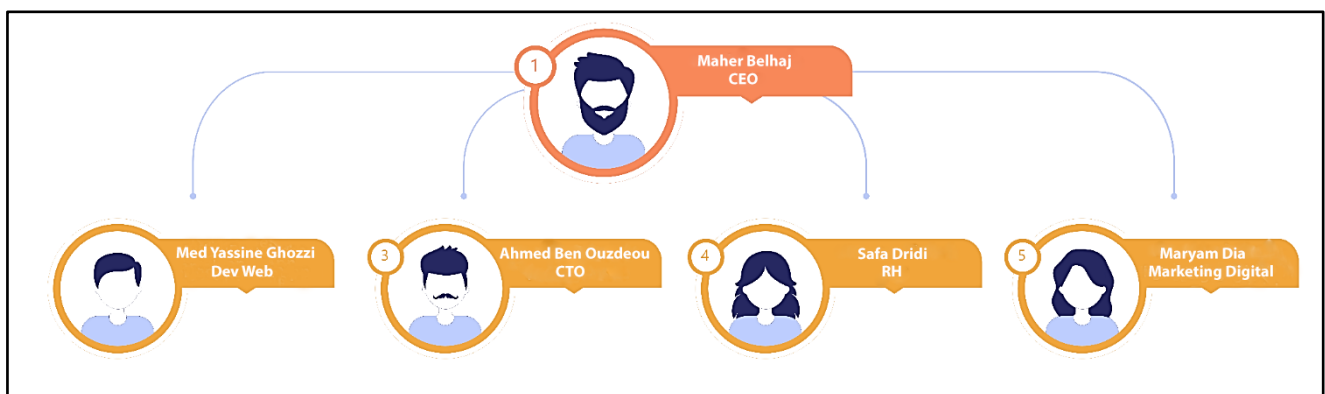


Figure 1 : L'équipe administrative d'Aryason consulting

1.2.3 Objectifs

Aryason Consulting comprend le développement de solutions informatique permettant d'optimiser la qualité de la gestion de l'approvisionnement à l'échelle internationale. Cette dernière vise avant tout à améliorer sa notoriété, son image de marque ainsi qu'à renforcer la fidélité de ses clients.

2 Présentation du projet

L'objectif de cette partie est de présenter le projet, objet de notre rapport. Nous nous intéresserons d'abord à l'étude de l'existant. Nous finirons par présenter la solution proposée.

2.1 Problématique

L'enjeu principal de notre projet est de trouver un niveau de surveillance optimal. De plus, les difficultés associées à une gestion efficace du traitement et du stockage des données, ainsi qu'à la complexité inhérente à la manipulation des bases de données, mettent en évidence l'importance d'améliorer ces aspects afin de garantir la réussite du projet.

2.2 Etude de l'existant

L'étude de l'existant est une étape primordiale qui permet de définir les forces et les faiblesses d'un produit afin de déterminer les besoins du client, et de les prendre en considération lors de la conception et de la réalisation de notre application.

2.3 Description & Critique de l'existant

Application : ARYANEGO (version bureautique)

Il s'agit d'une solution qui permet de gérer tous les processus de vente et de location des pylônes, notamment le suivi des commandes, la gestion des stocks, l'effectifs et bien d'autres encore. Toutefois, tout comme toute solution technologique, elle présente des avantages et des inconvénients.

Points faibles :

- ✖ Consommation de beaucoup de ressources.
- ✖ Système de sécurité non standard, complexe et mal adapté.
- ✖ Nécessité d'une installation sur chaque appareil.
- ✖ Utilise des fichiers Excel en tant que base de données.
- ✖ Très lente.

2.4 Solution proposée

Pour satisfaire les exigences de notre client, nous envisageons de créer une version web. Cette approche présente de nombreux avantages, tels qu'une meilleure accessibilité, une utilisation facilitée depuis divers appareils et lieux, une configuration plus adaptable, ainsi qu'une mise à jour plus rapide des fonctionnalités.

3 Méthodologie de travail

Pour éviter tout dépassement de délais et garantir la qualité du projet, il est indispensable d'adopter une méthodologie de développement est indispensable.

3.1 Comparaison entre les méthodes agiles et les méthodes traditionnelles

3.1.1 Agiles

Les méthodes agiles donnent la priorité au développement itératif et incrémental, tandis que les sprints permettent une adaptation rapide aux changements et assurent une livraison continue de petites fonctionnalités. Ils visent également à favoriser une coopération et une réactivité continues.

3.1.2 Traditionnelles

Les méthodes conventionnelles suivent un modèle séquentiel et linéaire, nécessitant la réalisation de phases telles que l'analyse, la conception, le développement, les tests, le déploiement et l'étape suivante. Le projet est généralement planifié à l'avance et sa rigidité peut rendre difficile l'intégration des changements.

3.1.3 Choix de méthode

Pour garantir une adaptation rapide aux changements avec une livraison continue et une amélioration constante de la valeur nous avons opté pour les méthodes agiles.

3.2 Comparaison entre Scrum et Kanban

3.2.1 Scrum

Scrum se distingue par des sprints avec d'une durée fixe permettant de livrer des fonctionnalités en constante évolution. En planifiant les sprints et en effectuant des revues, il est possible de gérer de manière prévisible le développement.

3.2.2 Kanban

Kanban utilise un tableau visuel pour gérer le flux de travail sans définir de sprints. Il favorise la flexibilité en permettant aux équipes d'ajuster les priorités en temps réel. Les limites de travail en cours aident à maintenir un flux de travail fluide et optimisé, adapté aux changements et aux besoins immédiats.

3.2.3 Choix de méthode

Nous avons opté pour la méthode de développement Agile pour garantir un niveau de qualité optimal dans la création de notre plateforme ERP. Cette approche collaborative et itérative permet à notre équipe de travailler de concert vers des objectifs précis, en minimisant les efforts et en respectant les délais impartis. En choisissant l'Agile, nous bénéficions de nombreux avantages tels que la flexibilité et la réactivité face aux changements, grâce à des cycles de développement courts. De plus, cette méthode nous permet de combiner théorie et pratique de manière efficace, en se rapprochant au mieux des besoins réels du projet. En résumé, l'adoption de l'approche Agile pour le développement de notre plateforme se révèle à la fois simple et efficace, comme illustré dans la **Figure 2**, où notre équipe se réunit quotidiennement

pour évaluer les progrès et planifier les étapes à venir.



Figure 2 : Processus de développement utilisant la méthodologie Scrum

Conclusion

Ce chapitre a abordé la présentation de l'entité d'accueil, du projet et la méthodologie adoptée. Le deuxième chapitre se concentrera sur les acteurs impliqués dans le projet et leurs besoins spécifiques. La présentation du backlog du projet ainsi que la planification des sprints et des livraisons seront réalisées à la fin de ce chapitre.

CHAPITRE 2

ANALYSE ET SPÉCIFICATION DES BESOINS

Introduction

Ce chapitre marque le début de notre projet en se concentrant initialement sur les parties prenantes impliquées dans notre système. Nous procéderons ensuite à l'analyse détaillée des exigences fonctionnelles et non fonctionnelles. Ces besoins seront ensuite formalisés à travers des diagrammes de cas d'utilisation, détaillant les scénarios possibles pour chaque acteur impliqué. Enfin, nous conclurons par la planification des sprints et l'exécution du projet selon les principes de la méthodologie Agile, assurant ainsi une gestion efficace et itérative de notre développement.

1 Identification des acteurs

Tout système interactif se doit d'assurer une interaction fluide et efficace avec ses utilisateurs, qu'ils soient humains ou non humains. Chaque acteur incarne le rôle d'une entité externe interagissant avec le système à travers diverses interfaces. Nous avons identifié les acteurs suivants :

- **Administrateur** : C'est la personne responsable de la gestion globale de notre application, en assurant que tout fonctionne efficacement.
- **Négociateur** : C'est le responsable d'un ou plusieurs sites avec leurs portefeuilles en fonction des prestations.
- **Client** : Il s'agit de la personne responsable du suivi des portefeuilles de ses sites, en s'assurant que les services répondent à ses besoins et attentes.

2 Analyse des besoins

L'analyse des besoins est réalisée au début, qui marque le début de notre processus de développement. Cette étape consiste à identifier les besoins fonctionnels et non fonctionnels initiaux, principalement à l'aide des descriptions textuelles et des diagrammes simplifiés.

2.1 Les besoins fonctionnels de l'application

Les besoins fonctionnels correspondant aux fonctionnalités attendues par l'utilisateur de l'application. Ils doivent répondre aux exigences du futur système qui doit accomplir ces fonctions et satisfaire ces besoins.

Notre plateforme doit permettre à l'administrateur, aux négociateurs et aux clients de réaliser leurs tâches spécifiques efficacement comme suit :

Gestion des utilisateurs et ses rôles

- Créer des rôles.
- Créer des utilisateurs et affectation d'un rôle.
- Consulter à la liste des utilisateurs.
- Modifier les utilisateurs.
- Connecter aux comptes utilisateurs.
- Recherche et filtrage des utilisateurs par des critères.
- Changer les mots de passe des comptes.

Gestion des sites

- Créer des sites.
- Consulter à la liste des sites.
- Modifier les sites.
- Recherche et filtrage des sites par des critères.

Gestion des objectifs

- Créer des objectifs
- Modifier un objectif.
- Consulter à la liste des objectifs
- Suivre les primes des négociateurs.
- Filtrage des objectifs par le négociateur, le mois et l'année.

Gestion des clients

- Créer des clients.
- Modifier un client.
- Consulter à la liste des clients.
- Créer un compte client.

Manipuler des portefeuilles

- Modifier la prestation de portefeuille.
- Modifier le portefeuille.

- Consulter à la liste des portefeuilles.
- Filtrage des portefeuilles selon des critères.
- Consulter à la liste des visites.
- Consulter à la liste d'historique d'appels.
- Exporter les portefeuilles.

Suivre les logs

- Consulter la liste des logs.
- Exporter la liste des logs.

Suivre le tableau de bord

- Suivre les changements d'état du site.
- Consulter l'historique de d'appels des négociateurs.
- Filtrer les sites selon les priorités.

2.2 Les besoins non-fonctionnels de l'application

Ces besoins décrivent les contraintes auxquelles notre plateforme est soumise pour assurer son bon fonctionnement.

- **Ergonomie** : Notre plateforme doit être facile à utiliser et intuitive pour garantir une expérience utilisateur optimale.
- **Sécurité** : L'accès à notre plateforme est limité aux utilisateurs disposant d'un compte actif, ce qui assure un niveau de sécurité élevé. De plus, les rôles et les autorisations accordés à chaque utilisateur sont soigneusement définis pour renforcer cette sécurité.
- **Maintenabilité** : La conception de l'application doit permettre une évolutivité aisée, ainsi qu'une maintenance cohérente à faible coût. Cela inclut la mise en œuvre de tests automatisés pour garantir la stabilité du code, la surveillance continue via un tableau de bord pour détecter et réagir rapidement aux problèmes, ainsi une documentation claire pour faciliter la compréhension et la maintenance du système.
- **Performance** : Notre plateforme doit être rapide et réactive pour garantir une expérience utilisateur fluide. Cela inclut plusieurs techniques qu'ils sont
 - Faire une consommation sélective des données, en ne récupérant que celles nécessaires à chaque demande spécifique.
 - L'implémentation de la pagination côté backend, nous avons optimisé la gestion des données en limitant la quantité des données d'informations traitées
 - L'utilisation de technique de Lazy Loading dans Angular, permettant des charger les modules et les fonctionnalités uniquement au moment où ils sont requis par l'utilisateur.

- **La convivialité** : Notre plateforme doit offrir une expérience utilisateur intuitive et conviviale pour faciliter l'utilisation de toutes les fonctionnalités proposées.

3 Diagramme de cas d'utilisation global de l'application

À ce stade, il est essentiel de spécifier toutes les opérations effectuées par le système qui ont un impact perceptible sur les utilisateurs. Le diagramme de cas d'utilisation global de notre plateforme représentée par la **Figure 3** illustre les grandes fonctionnalités de l'application en utilisant des diagrammes sans entrer dans les détails des interactions entre les acteurs et le système [3].

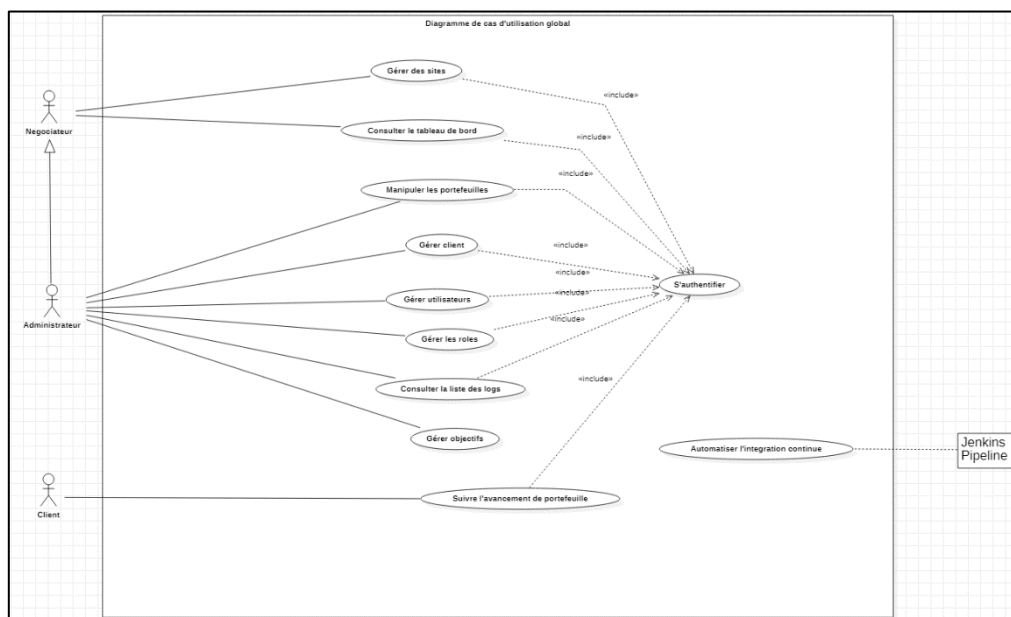


Figure 3 : Diagramme de cas d'utilisation global de << ARYANEGO >>

4 Analyse globale

Dans cette section, nous allons présenter des représentations sous forme d'arbres des différentes collections, ainsi que l'architecture physique et logique de notre application, illustrées à l'aide de diagrammes.

4.1 Architectures de l'application

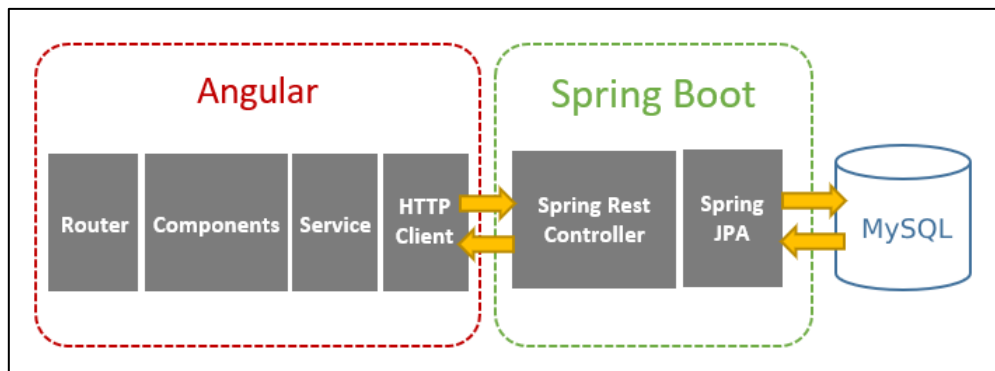
4.1.2 Architecture Logique

Voici les principaux composants nécessaires dans notre application selon l'architecture Angular illustrée dans la **Figure 4** :

- **Router** : Cet élément facilite la navigation entre les différentes sections de l'application en utilisant des URLs. Il permet également de définir des paramètres de route et de créer des routes imbriquées.
- **Service** : Les services sont des classes qui permettent de partager des données et des fonctionnalités entre différents éléments de l'application. Ils sont utiles pour interagir avec des APIs externes ou pour gérer le stockage local de données.
- **HttpClient** : Ce module Angular permet d'envoyer des requêtes HTTP à un serveur distant et de recevoir les réponses correspondantes. Il est souvent utilisé dans les services pour récupérer des données depuis une API.
- **Components** : Les composants sont les éléments fondamentaux d'une application Angular. Ils sont utilisés pour créer les différentes vues de l'application et peuvent être imbriqués les uns dans les autres pour former une structure hiérarchique. Chaque composant est composé d'un fichier template HTML, d'un fichier de style CSS et d'une classe TypeScript.

Dans Spring boot, les composants nécessaires pour notre application incluent :

- **Spring REST Controller** : Il s'agit d'un composant de Spring MVC qui permet de gérer les requêtes HTTP et de produire des réponses HTTP, souvent en format JSON ou XML.
- **Spring Data JPA** : C'est un projet Spring visant à simplifier l'accès aux données dans les



applications. Il facilite la mise en œuvre de la couche d'accès aux données.

Figure 4 : Architecture logique Angular et Spring Boot

4.2.3 Architecture physique globale

L'architecture Physique de « ARYANEGO » est représentée par la **Figure 5**.

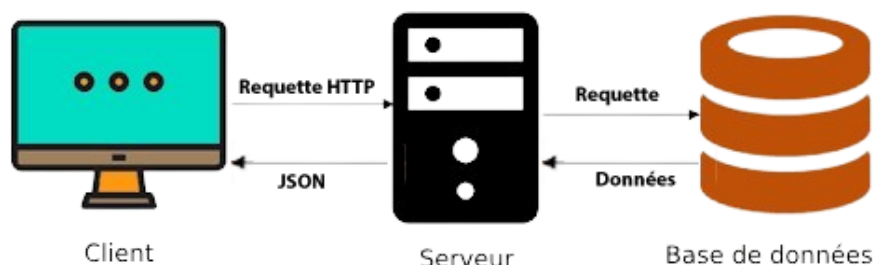


Figure 5 : Architecture physique globale

5 Coordination de projet en utilisant la méthodologie Scrum

Dans la continuité de notre méthode Scrum, nous aborderons dans l'attribution des tâches au sein de l'équipe, l'élaboration des sprints et la gestion du backlog

5.1 Répartition du travail

Le **tableau 1** montre la répartition du travail pour le projet actuel. Cette répartition vise à accroître l'efficacité et à attribuer des tâches spécifiques à chaque membre de l'équipe. Pour atteindre efficacement nos objectifs, il est important de définir clairement les rôles et les responsabilités. Examinons de plus près la répartition des tâches dans le tableau ci-dessous.

Rôle	Acteur	Missions
Product owner	Maryam Dia	- Définir le besoin. - Définir l'ordre dans lequel les fonctionnalités seront développées.
Scrum master	Mouhamed Hammoud	Assurer le bon déroulement du projet.
Team lead	Ben Ouzdeou Ahmed	Donner des instructions et surveiller les performances.
Equipe de développement	Mohamed Dhahbi	Développement de l'application.

Tableau 1 : Répartition du travail

5.2 Backlog du produit

Le **Tableau 2** présente le backlog produit, qui est une liste ordonnée des fonctionnalités et tâches à réaliser [4]. Examinons de plus près les éléments inclus dans le backlog.

ID	Module	User story ID	User Story	Priorité	Estimation(j)
1	Gestion des utilisateurs et ses rôles	1.1	En tant qu'utilisateur, je souhaite authentifier pour accéder à mon compte	Haute	7 jours
		1.2	En tant qu'administrateur, je souhaite créer des rôles.	Haute	2 jours
		1.3	En tant qu'administrateur, je souhaite créer des utilisateurs.	Haute	4 jours
		1.4	En tant qu'administrateur, je souhaite consulter la liste des utilisateurs.	Haute	2 jours
		1.5	En tant qu'administrateur, je souhaite chercher et filtrer les utilisateurs selon leurs noms, email, date d'entrée et ses rôles.	Haute	6 jours
		1.6	En tant qu'un utilisateur, je souhaite modifier le mot de passe de compte.	Haute	5 jours
2	Gestion des Sites	2.1	En tant qu'administrateur, je souhaite créer des sites	Haute	5 jours
		2.2	En tant qu'administrateur et Négociateur, je souhaite consulter la liste des sites	Haute	3 jours
		2.3	En tant qu'administrateur je souhaite modifier un site	Haute	4 jours

CHAPITRE 2 : ANALYSE ET SPÉCIFICATION DES BESOINS

		2.4	En tant qu'administrateur je souhaite supprimer un site avec ses prestations.	Haute	3 jours
		2.5	En tant qu'administrateur, je souhaite chercher et filtrer les sites selon le nom, le code, les départements, les parcs, les responsables de site et la date de création.	Haute	5 jours
3	Gestion des clients	3.1	En tant qu'administrateur, je souhaite créer des clients.	Haute	5 jours
		3.2	En tant qu'administrateur, je souhaite consulter à la liste des clients.	Haute	2 jours
		3.3	En tant qu'administrateur, je souhaite modifier les informations du client.	Haute	6 jours
		3.4	En tant qu'administrateur, je souhaite créer un compte à travers les informations de client.	Haute	7 jours
4	Gestion des objectifs	4.1	En tant qu'administrateur, je souhaite créer des objectifs.	Haute	5 jours
		4.2	En tant qu'administrateur, je souhaite consulter à la liste des objectifs.	Haute	2 jours
		4.3	En tant qu'administrateur, je souhaite modifier un objectif.	Haute	4 jours
		4.4	En tant qu'administrateur, je souhaite suivre les primes des négociateurs	Haute	7 jours
		4.5	En tant qu'administrateur, je souhaite filtrer les objectifs selon les négociateurs, le mois et l'année.	Haute	4 jours
5	Manipulation des portefeuilles	5.1	En tant qu'administrateur, je souhaite modifier l'état de prestation de site.	Haute	1 jours
		5.2	En tant qu'administrateur, je souhaite modifier les prestations de site.	Haute	16 jours

CHAPITRE 2 : ANALYSE ET SPÉCIFICATION DES BESOINS

		5.3	En tant qu'administrateur, je souhaite ajouter des visites.	Haute	2 jours
		5.4	En tant qu'administrateur, je souhaite consulter la liste des visites.	Haute	2 jours
		5.5	En tant qu'administrateur, je souhaite consulter la liste d'historisation des appels.	Haute	2 jours
		5.6	En tant qu'administrateur, je souhaite chercher des portefeuilles.	Haute	2 jours
		5.7	En tant qu'administrateur, je souhaite exporter les portefeuilles.	Haute	2 jours
		5.8	En tant qu'administrateur et client, je souhaite consulter la liste des portefeuilles.	Haute	2 jours
6	Suivre les logs	6.1	En tant qu'administrateur, je souhaite, consulter la liste des logs	Haute	2 jours
		6.2	- En tant qu'administrateur, je souhaite exporter les logs dans un fichier Excel.		2 jours
7	Suivre les statistiques de plateforme	7.1	En tant qu'administrateur, je souhaite consulter au statistique de performance.	Haute	5 jours
		7.2	En tant qu'administrateur, je souhaite, suivre les changements d'état du site.	Haute	2 jours
		7.3	En tant qu'administrateur, je souhaite Filtrer la production par prestation, par et négociateur.	Haute	5 jours
		7.4	En tant qu'administrateur, je souhaite filtrer les sites selon les priorités.	Haute	1 jours
		7.5	En tant qu'administrateur, je souhaite consulter l'historique de d'appels des négociateurs.	Haute	2 jours
		7.6	En tant qu'administrateur, je souhaite consulter aux emplacements des sites.	Haute	6 jours

		7.7	En tant qu'administrateur, je souhaite filtrer les contrats de négociation.	Haute	2 jours
8	Mise en place l'automatisation	8.1	En tant qu'Administrateur, je souhaite suivre le résultat de pipeline.	Haute	14 jours

Tableau 2 : Backlog du produit

5.3 Planification des sprints

En se basant sur le backlog produit, nous avons planifié notre Projet en plusieurs sprints [5], comme présenté dans le **Tableau 3** ci-dessous.

Sprints	Durée(semaine)
Sprint 1 : Gestion des utilisateurs et les rôles.	3 semaines
Sprint 2 : Gestion des sites.	3 semaines
Sprint 3 : Gestion des clients.	3 semaines
Sprint 4 : Gestion des objectifs.	3 semaines
Sprint 5 : Manipulation des portefeuilles des sites.	3 semaines
Sprint 6 : Implémentation des logs et du tableau de bord.	3 semaines
Sprint 7 : Mise en place l'automatisation	3 semaines

Tableau 3 : Planification des sprints

5.4 Planification des releases

Le **Tableau 4** présente la planification des releases pour le projet. Les releases regroupent les sprints qui incluent des fonctionnalités et des améliorations spécifiques. Examinons de plus près la planification des releases dans le tableau ci-dessous.

Releases	Sprints
Release 1 : Gestion des utilisateurs, les rôles, les sites et les clients.	Sprint 1, Sprint 2, Sprint 3
Release 2 : Gestion des objectifs, manipulation des portefeuilles et implémentation des logs et du tableau de bord.	Sprint 4, Sprint 5, Sprint 6
Release 3 : Mise en place de l'automatisation.	Sprint 7

Tableau 4 : Planification des releases

5.5 Diagramme de Gantt

La **Figure 6** présente le diagramme de Gantt de notre stage [4].

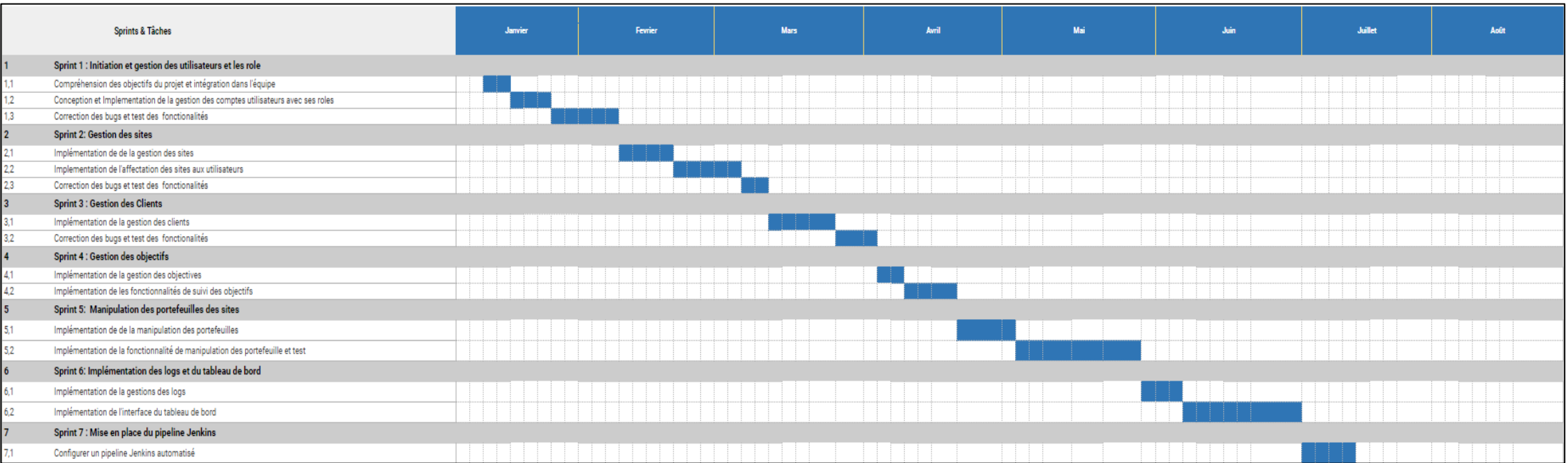


Figure 6 : Diagramme de Gantt de notre stage

6 Étude des choix techniques et outils

Nous présenterons dans cette section les outils et technologies qui ont été utilisés pour le développement de notre plateforme.

6.1 Choix des Framework du Frontend

Nous avons opté pour Angular 13 comme technologie côté client, comme illustré dans la Figure 7. Angular est une plateforme de développement web open-source prisée, soutenue principalement par Google et sa communauté de développeurs. Fondé sur TypeScript, un langage de programmation développé par Microsoft, Angular offre une structure robuste pour la création d'interfaces web fonctionnelles.



Figure 7 : Logo Angular

6.2 Choix de Framework Backend

Nous avons choisi Spring Boot comme technologie coté serveur, il s'agit d'une technologie open source basé sur Spring conçu pour simplifier le processus de création des micro-services et d'application personnalisés en Java.



Figure 8 : Logo Spring Boot

6.3 Choix de la base de données

Nous avons opté MySQL, **Figure 9**, qui est un système de gestion de base de données relationnelles open source, il utilise la langage SQL interagir avec les données stockées dans des tables.



Figure 9 : Logo MySQL

6.4 Docker

Est une plateforme permettant de lancer certaines applications dans des conteneurs logiciels lancée en 2013.



Figure 10 : Logo Docker

6.5 Jenkins

Est un outil de serveur d'automatisation open source. Il facilite l'intégration et la livraison continue en aidant à lancer les phases de développement logiciel qui incluent la construction, des tests et le déploiement. Il est écrit en Java.



Figure 11 : Logo Jenkins

6.6 Environnement logiciel

- **GitHub** : GitHub, **Figure 12**, est une plateforme de développement collaboratif qui permet aux équipes de collaborer sur des projets logiciels.



Figure 12 : Logo GitHub

- **Intellij** : Il s'agit d'un logiciel de développement informatiques basé sur la technologie Java. Il a été développé par JetBrains et disponible en deux versions l'une communautaire, open source sous licence Apache 2 et l'autre est maintenue sous un licence commerciale.

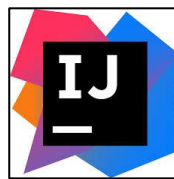


Figure 13 : Logo IntelliJ

- **StarUML** : StarUML, **Figure 14**, est un outil de modélisation UML puissant et facile à utiliser. Il permet aux développeurs de créer des diagrammes UML pour exprimer, concevoir et documenter avec précision la structure et le comportement d'un système.



Figure 14 : Logo StarUML

- **Visual Studio Code** : Visual Studio Code, **Figure 15**, est un éditeur de code source simple et gratuit, développé par Microsoft. Il offre une interface conviviale et de nombreuses fonctionnalités, telles que la visualisation de syntaxe, le débogage, les

extensions, un développement et une programmation faciles dans différents langages

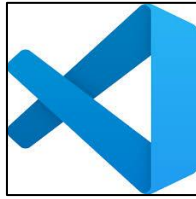


Figure 15 : Logo Visual Studio Code

Conclusion

La clarté de nos besoins spécifiques nous a permis de développer une meilleure vision du sujet et une compréhension plus approfondie des tâches à accomplir. Cela nous a également permis d'anticiper les problèmes potentiels et de rechercher des solutions pour les surmonter. Dans ce chapitre, nous avons tenté de présenter les besoins fonctionnels et non fonctionnels du sujet, ainsi que les diverses fonctionnalités du backlog. Dans le prochain chapitre, nous aborderons les releases du projet, en détaillant les fonctionnalités. Nous approfondirons également l'analyse du backlog en décomposant les fonctionnalités en tâches spécifiques et en définissant les critères d'acceptation.

CHAPITRE 3

RELEASE 1 : GESTION DES UTILISATEURS, LES RÔLES, LES SITES ET LES CLIENTS

Introduction

Ce chapitre présente le premier release du projet intitulé "Implémentation des interfaces et la gestion des utilisateurs, les rôles et les clients". Ce release se compose de trois sprints distincts, le premier est axé sur l'implémentation des interfaces, le deuxième sur la gestion des utilisateurs avec leurs rôles et le troisième sprint pour la gestion des clients. Ces trois sprints se dérouleront sur une période totale de trois mois.

1 Backlogs des releases

Le backlog de sprint est composé des éléments sélectionnés à partir du backlog de produit. Ces éléments sont généralement convenus lors de la planification de sprint et représentent le résultat principal de cette dernière. Les **Tableaux 3.1, 3.2 et 3.3** exposeront les tâches qui ont été effectuées aux trois premiers sprints.

1.1 Backlog du sprint 1

ID	User Story	ID tâches	Tâches	Priorité
1.1	En tant qu'un utilisateur, je souhaite authentifier pour accéder à mon compte	1.1	▪ Implémenter la logique d'authentification dans la partie backend. ▪ Faire un test de la fonctionnalité d'authentification. ▪ Implémenter l'interface de connexion côté client et intégrer l'appel à la logique métier côté serveur.	Haute
		1.2		
		1.3		

CHAPITRE 3 : RELEASE 1 : GESTION DES UTILISATEURS, LES ROLES, LES SITES ET LES CLIENTS

1.2	En tant qu'administrateur, je souhaite créer des rôles.	2.1	- Préparer l'interface d'ajout d'un rôle. - Implémenter la logique coté serveur pour la gestion des rôles. - Consommer l'api de création des rôles.	Haute
		2.2		
		2.3		
1.3	En tant qu'administrateur, je souhaite créer des utilisateurs	3.1	- Préparer l'interface d'ajout d'un utilisateur. - Implémenter la logique coté serveur pour la création d'un utilisateur. - Consommer l'api d'ajout d'un utilisateur.	Haute
		3.2		
		3.3		
1.4	En tant qu'administrateur, je souhaite consulter la liste des utilisateurs.	4.1	- Préparer l'interface de tableau des utilisateurs. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		4.2		
		4.3		
1.5	En tant qu'administrateur, je souhaite chercher et filtrer les utilisateurs.	5.1	- Préparer la section de filtrage et recherche par les critères correspondant. - Consommer les apis de filtrage et de recherche.	Haute
		5.2		
1.6	En tant qu'un utilisateur, je souhaite modifier le mot de passe de mon compte.	6.1	- Préparer l'interface utilisateur pour la modification du mot de passe. - Implémenter la logique coté serveur pour la modification de mot de passe de compte utilisateur. - Implémenter la fonctionnalité de modification du mot de passe.	Haute
		6.2		
		6.3		
1.7	En tant qu'administrateur, je souhaite modifier un utilisateur.	7.1	- Préparer l'interface de modification d'un utilisateur. - Implémenter la logique coté serveur pour la modification. - Consommer l'api de modification d'un utilisateur.	Haute
		7.2		
		7.3		

Tableau 5 : Backlog du sprint 1

1.2 Backlog du sprint 2

ID	User Story	ID tâches	Tâches	Priorité
2.1	En tant qu'administrateur, je souhaite créer des sites.	1.1	- Préparer l'interface d'ajout d'un site. - Implémenter la logique coté serveur pour la création d'un site. - Consommer l'api d'ajout d'un site.	Haute
		1.2		
		1.3		
2.2	En tant qu'administrateur et négociateur, je souhaite consulter la liste des sites.	2.1	- Préparer l'interface de tableau des sites. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		2.2		
		2.3		
2.3	En tant qu'administrateur, je souhaite modifier un site.	3.1	- Préparer l'interface de modification d'un site. - Implémenter la logique coté serveur pour la modification. - Consommer l'api de modification d'un site.	Haute
		3.2		
		3.3		
2.4	En tant qu'administrateur, je souhaite supprimer un site avec tous ses prestations.	4.1	- Implémenter la logique coté serveur pour la suppression. - Consommer l'api de suppression d'un site.	Haute
		4.2		
2.5	En tant qu'administrateur, je souhaite chercher et filtrer les sites.	5.1	- Préparer la section de filtrage et recherche des sites. - Consommer l'api de filtrage des clients selon le nom, le code, les départements, les parcs, les responsables de site et la date de création.	Haute
		5.2		

Tableau 6 : Backlog du sprint 2

1.3 Backlog du sprint 3

ID	User Story	ID Tâches	Tâches	Priorité
3.1	En tant qu'administrateur, je souhaite créer des clients.	1.1	- Préparer l'interface d'ajout d'un client. - Implémenter la logique coté serveur pour la création d'un client. - Consommer l'api d'ajout d'un client.	Haute
		1.2		
		1.3		
3.2	En tant qu'administrateur, je souhaite consulter la liste des clients.	2.1	- Préparer l'interface de tableau des sites. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		2.2		
		2.3		
3.3	En tant qu'administrateur, je souhaite modifier les clients.	3.1	- Préparer l'interface de modification d'un client. - Implémenter la logique coté serveur pour la modification. - Consommer l'api de modification d'un client.	Haute
		3.2		
		3.3		
3.4	En tant qu'administrateur, je souhaite créer un compte à travers les informations de client.	4.1	- Préparer les sections relatives aux informations supplémentaires dans le formulaire d'ajout et modification d'un client. - Intégrer les informations supplémentaires dans le corps de la requête envoyée au backend dans la méthode de consommation de l'api de l'ajout et modification de client.	Haute
		4.2		

Tableau 7 : Backlog du sprint 3

2 Raffinement des diagrammes du Release 1

Dans cette section, nous allons nous concentrer sur les diagrammes raffinés de ce release.

2.1 Diagramme de cas d'utilisation « Gestion des utilisateurs et les rôles »

Le diagramme de cas d'utilisation détaillé de la partie Gestion des utilisateurs et ses rôles est représenté dans la **Figure 16** ci-dessous.

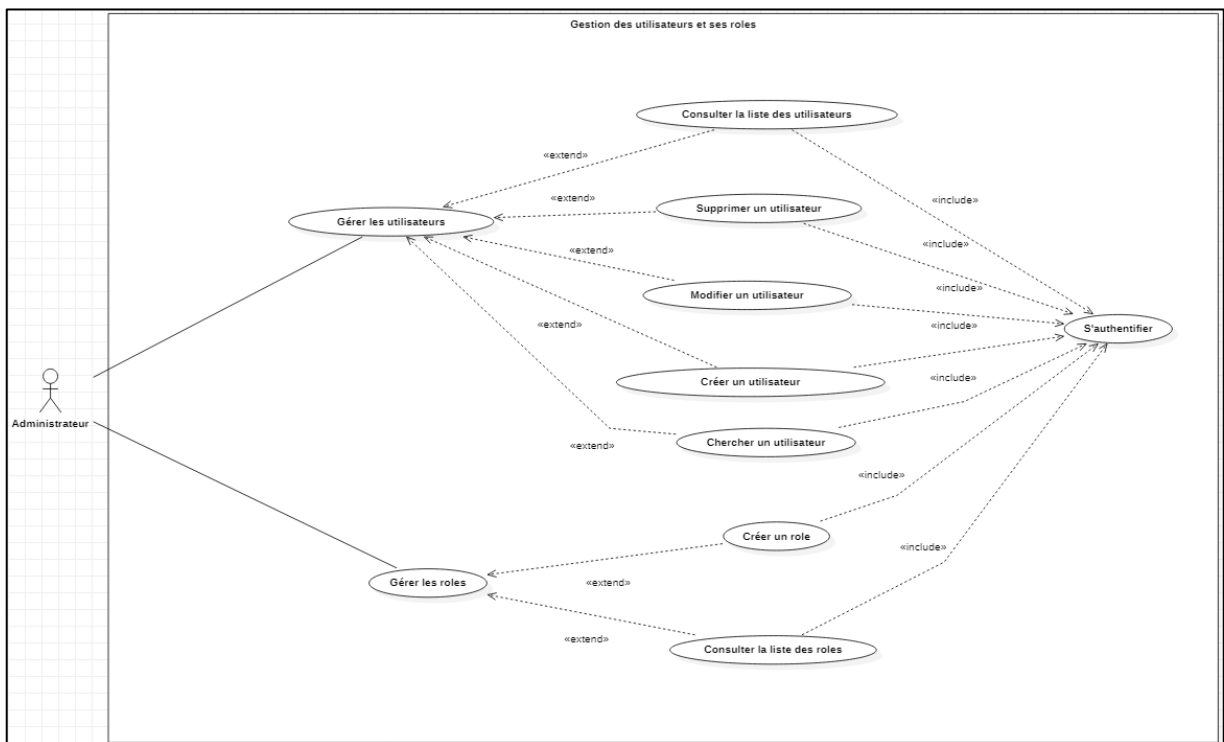


Figure 16 : Diagramme de cas d'utilisation « Gestion des utilisateurs et les rôles »

2.2 Diagramme de cas d'utilisation « Gestion des Sites »

Le diagramme de cas d'utilisation détaillé de la partie gestion des sites est représenté dans la **Figure 17**.

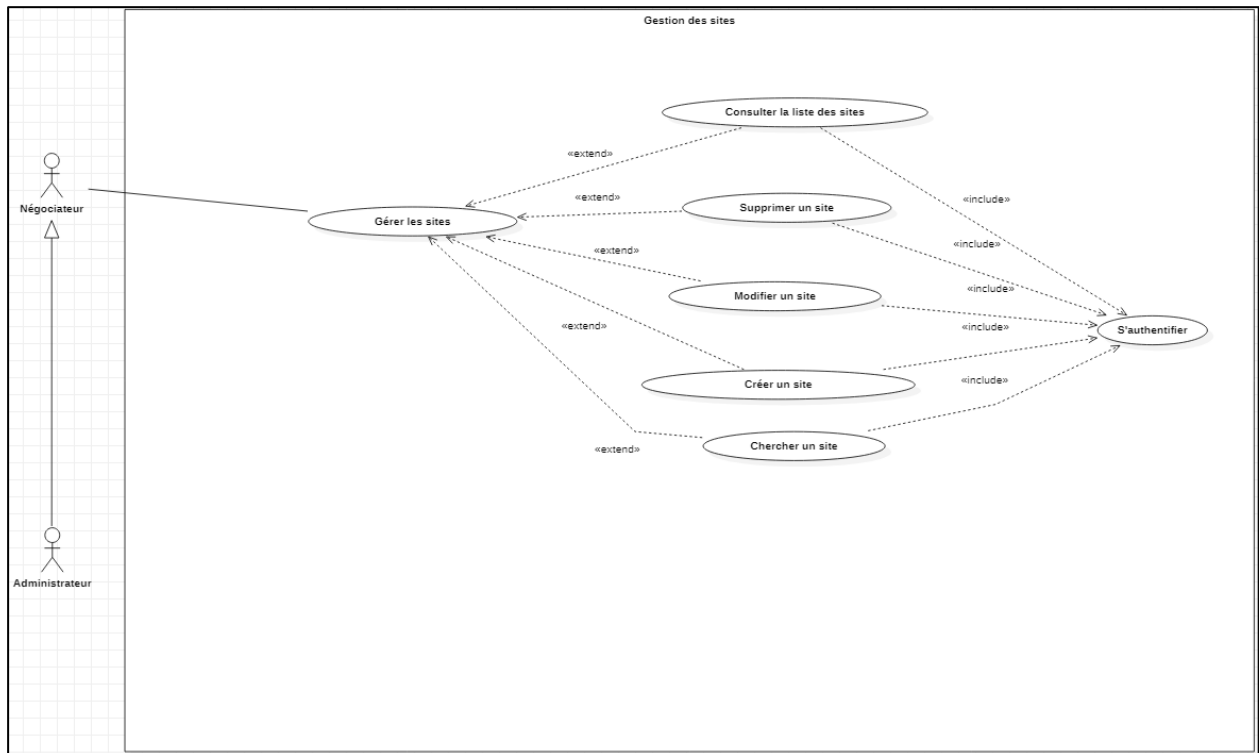


Figure 17 : Diagramme de cas d'utilisation « Gérer Sites »

2.3 Diagramme de cas d'utilisation « Gestion des Clients »

Le diagramme de cas d'utilisation détaillé de la partie Gestion des Clients est représenté dans la **Figure 18**.

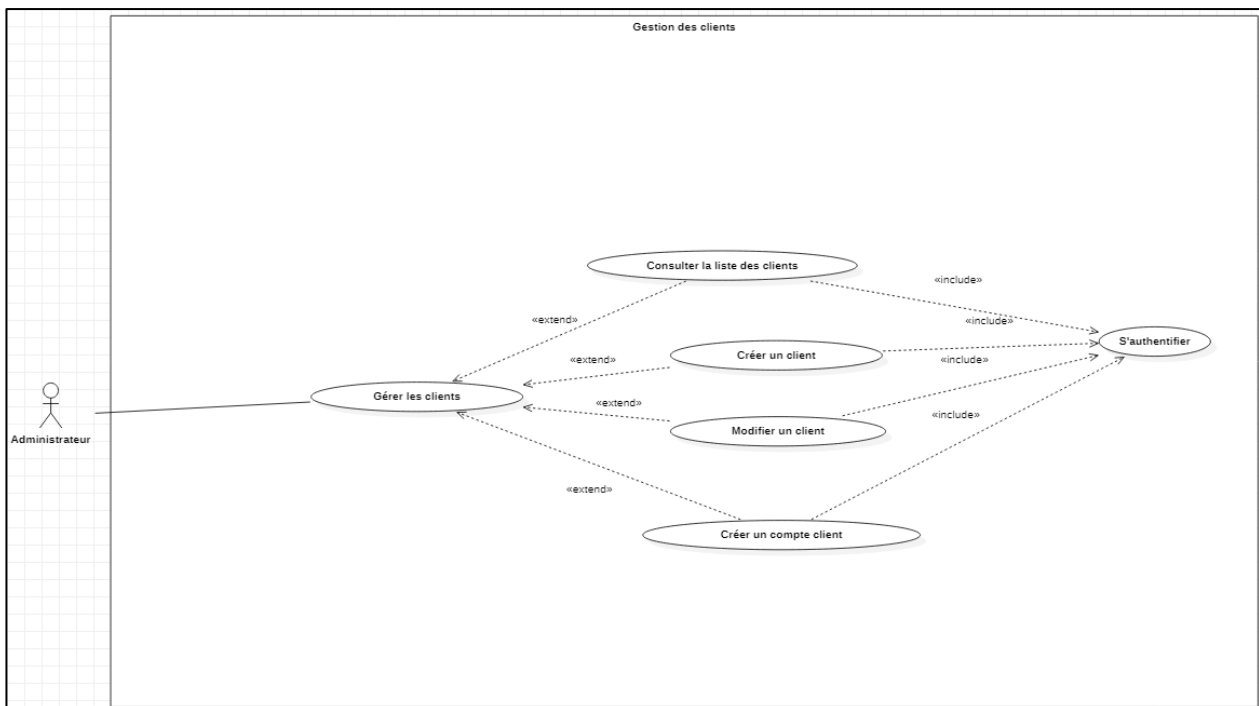


Figure 18 : Diagramme de cas d'utilisation « Gestion des clients »

2.4 Description d'un cas d'utilisation du Release 1

2.4.1 Description du cas « Ajouter un utilisateur »

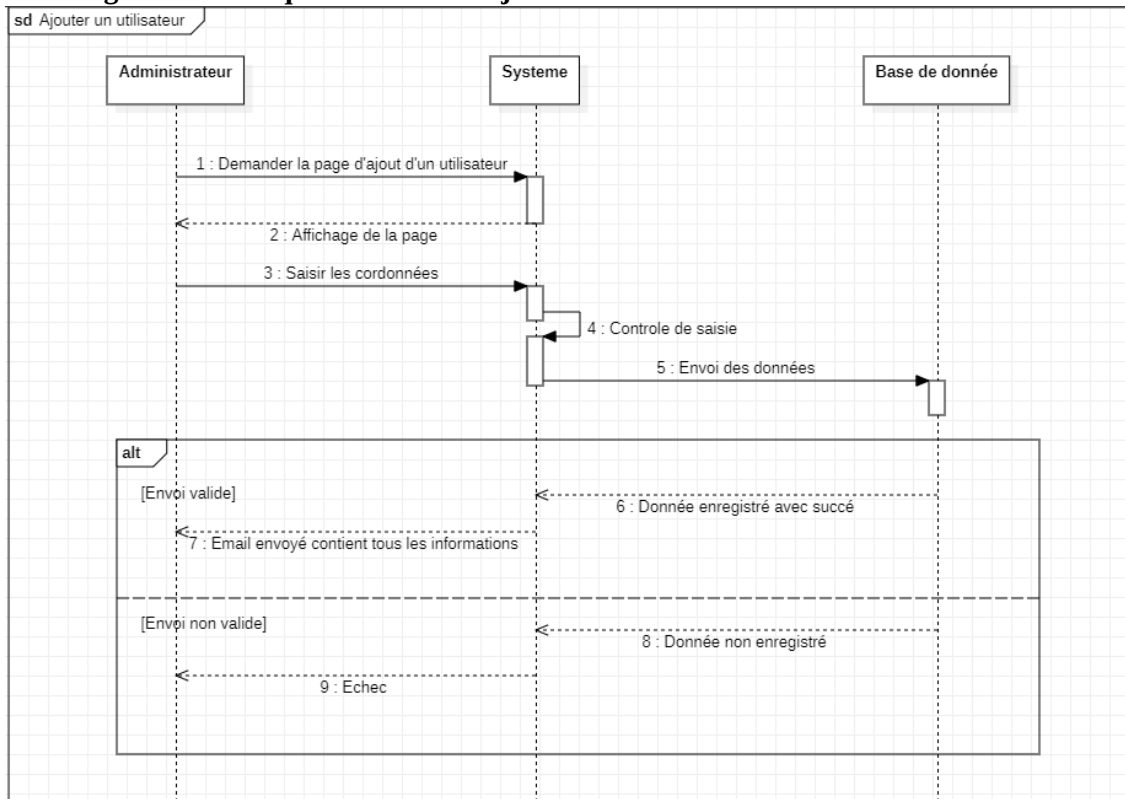
2.4.1.1 Description textuelle

Dans cette section, nous présentons la description et le diagramme de séquence du cas d'utilisation « Ajouter un utilisateur ». La description textuelle correspondante est présentée dans le **Tableau 8** ci-dessous.

Acteur	Administrateur
Pré condition	L'administrateur a déjà un compte sur « ARYANEGO »
Post condition	L'utilisateur est ajouté.
Scénario	1. L'Administrateur remplit le formulaire d'ajout d'utilisateur avec les informations nécessaires. 2. L'utilisateur est ajouté avec succès et un email envoyé contient les informations de compte.
Scénario alternatif	1. Affichage d'un message d'erreur si le formulaire n'est pas correctement rempli.

Tableau 8 : Description textuelle du cas « Ajouter un utilisateur »

3.5.1.2 Diagramme de séquence du cas « Ajouter un utilisateur »



La **Figure 19** présentée ci-dessous montre le processus d'ajout d'un prix pour le produit ainsi que son déroulement.

Figure 19 : Diagramme de séquence de cas « Ajouter un utilisateur »

4 Conception

4.1 Etablir Diagramme de classe du release 1

Le diagramme de classe nous permet d'appréhender la structure globale du système en identifiant les classes principales et leurs relations, sans entrer dans les détails spécifiques de chaque classe ou de leurs attributs. Il offre ainsi une vue d'ensemble de la conception de notre application. La **Figure 20** présente le diagramme de classe du release 1.

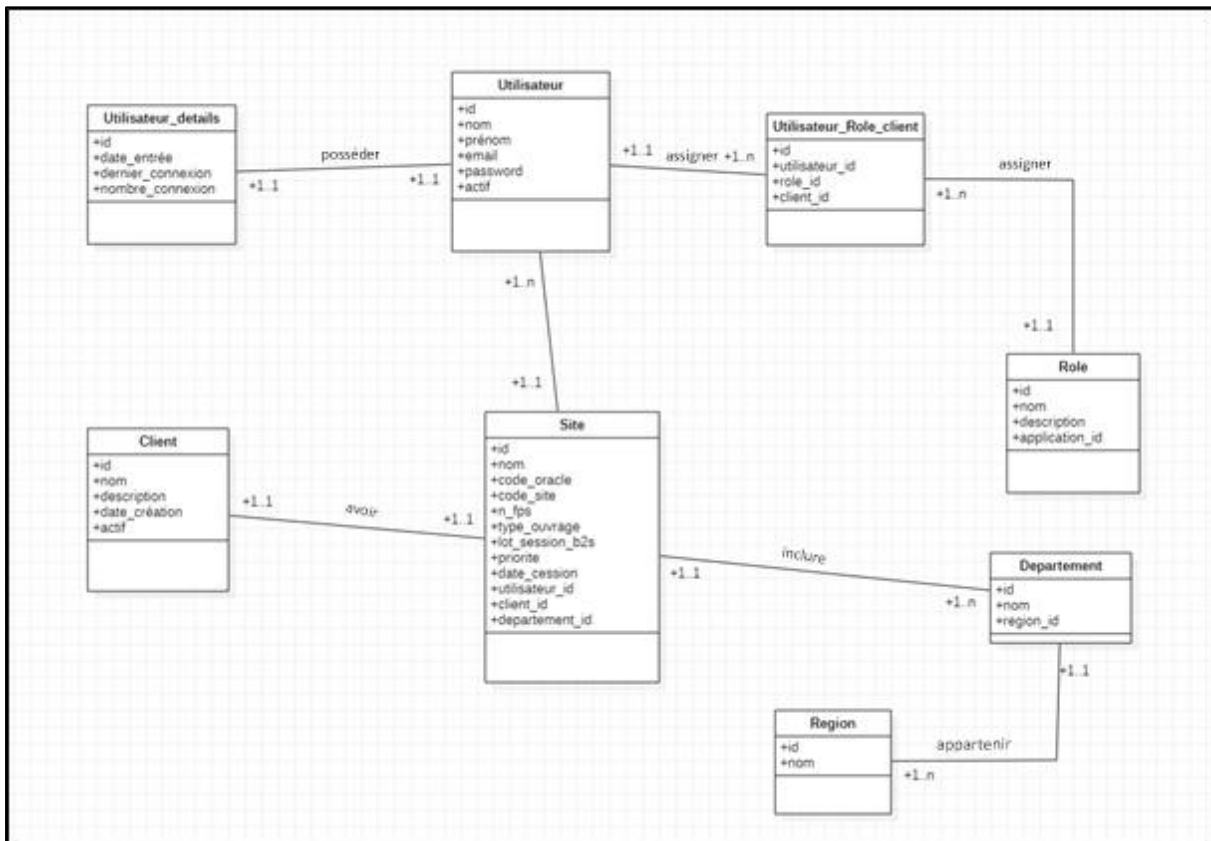


Figure 20 : Diagramme de classe de release 1

5 Réalisation

Dans cette section, nous présenterons quelques interfaces des parties traitées lors de ce release. Ces interfaces permettront d'illustrer concrètement les avancées réalisées dans le développement de notre application.

5.1 Gestion des utilisateurs et ses rôles

La **Figure 21** ci-dessous présente l'interface de connexion de la plateforme « ARYANEGO ».

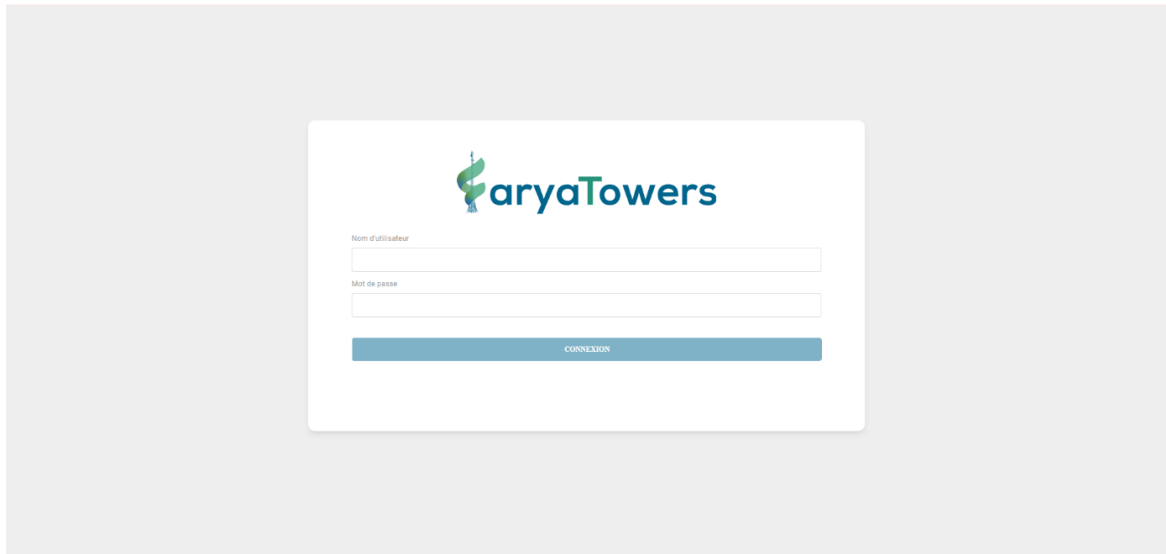


Figure 21 : Interface de connexion

CHAPITRE 3 : RELEASE 1 : GESTION DES UTILISATEURS, LES ROLES, LES SITES ET LES CLIENTS

La **Figure 22** ci-dessous présente l’interface de gestion des utilisateurs et ses rôles de plateforme « ARYANEGO ».

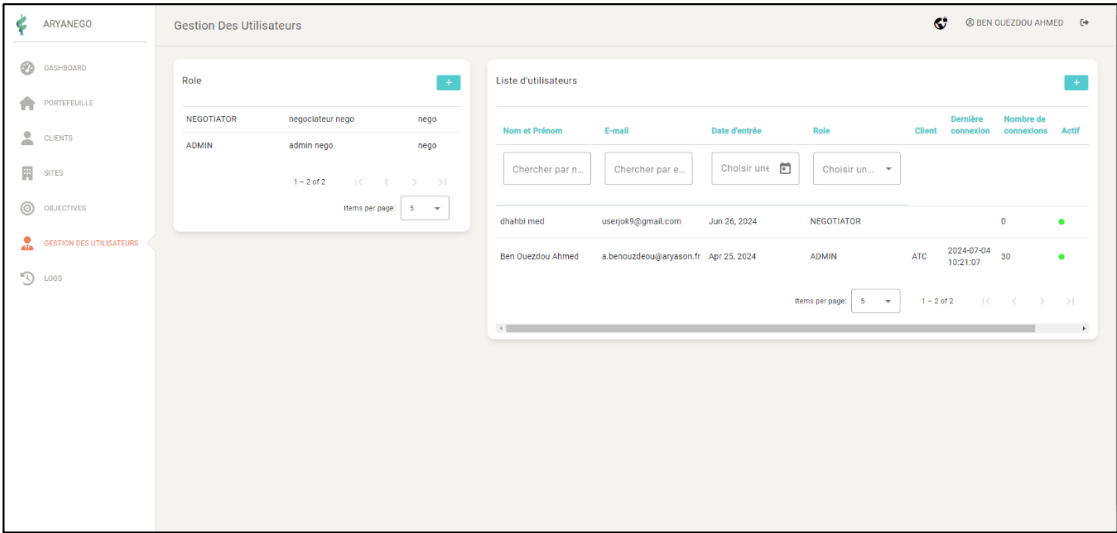


Figure 22 : Interface de gestion des utilisateurs et les rôles

La **Figure 23** ci-dessous présente l’interface d’ajout d’un utilisateur dans « ARYANEGO ».

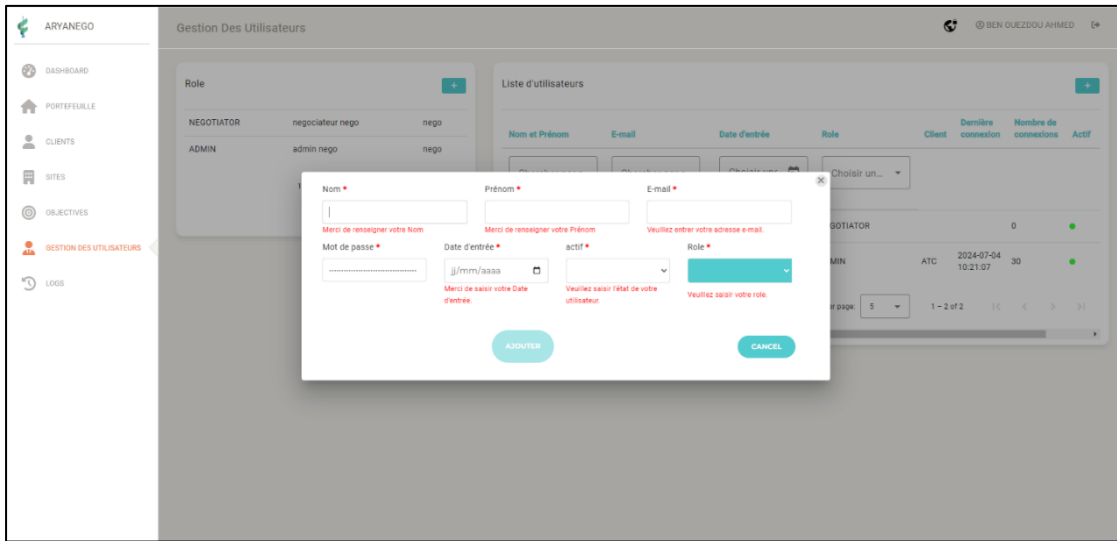


Figure 23 : Interface d'ajout d'un utilisateur

CHAPITRE 3 : RELEASE 1 : GESTION DES UTILISATEURS, LES ROLES, LES SITES ET LES CLIENTS

La **Figure 24** ci-dessous présente l'interface d'ajout d'un nouveau rôle dans « ARYANEGO ».

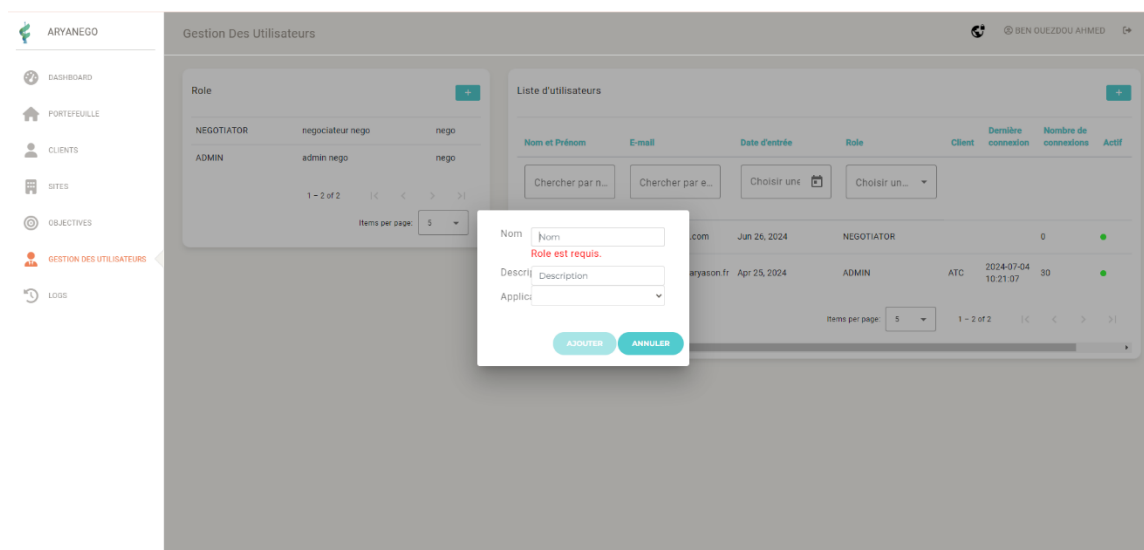


Figure 24 : Interface d'ajout d'un rôle

5.2 Gestion des sites

La **Figure 25** ci-dessous présente l'interface de gestion des sites dans « ARYANEGO ».

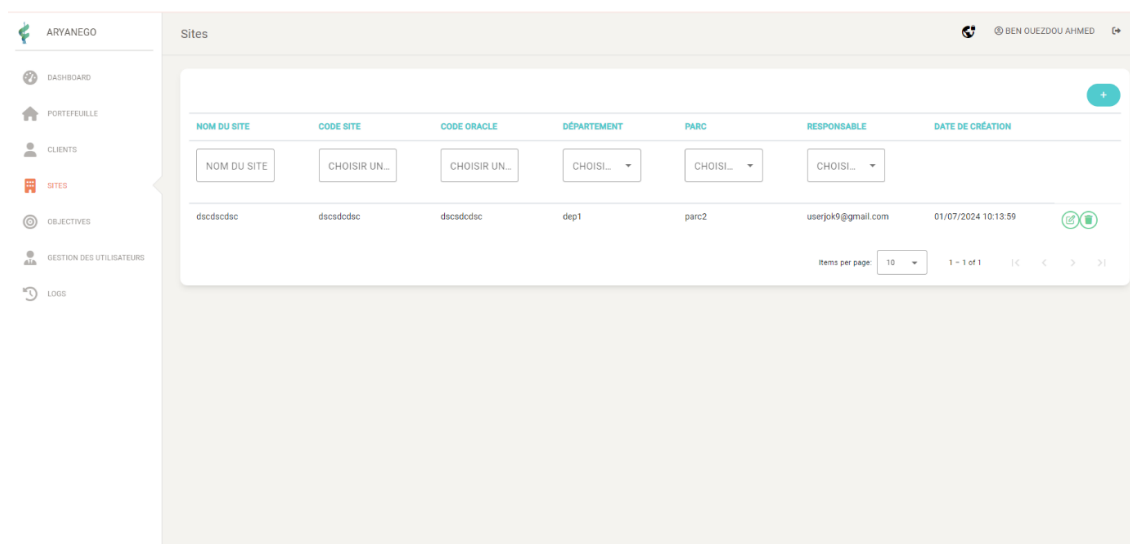


Figure 25 : Interface de gestion des sites

CHAPITRE 3 : RELEASE 1 : GESTION DES UTILISATEURS, LES ROLES, LES SITES ET LES CLIENTS

La **Figure 26** ci-dessous présente l'interface d'ajout des sites dans « ARYANEGO ».

The screenshot shows the 'Ajout d'un site' (Add site) form in the ARYANEGO application. The form is located on the 'Dashboard' page. It contains several input fields and dropdown menus for adding a new site. The fields are organized into three columns. The first column includes 'Client' (a dropdown menu), 'Code Strade' (a text field), 'Lot de cession S2S' (a text field), and 'Département' (a dropdown menu). The second column includes 'N°FP5' (a text field), 'Nom du site' (a text field), 'Priorité' (a dropdown menu), and 'Pays' (a dropdown menu). The third column includes 'Code site' (a text field), 'Type d'ouvrage' (a dropdown menu), 'Région' (a dropdown menu), and 'Négociateur' (a dropdown menu). A green 'AJOUTER' button is located at the bottom left of the form.

Figure 26 : Interface d'ajout d'un site

5.3 Gestion des Clients

La **Figure 27** ci-dessous présente l'interface de gestion des clients dans « ARYANEGO ».

The screenshot shows the 'Gestion des Clients' (Client Management) interface in the ARYANEGO application. It features a table with client data and a sidebar with navigation options. The table has the following columns: 'Code client', 'Raison sociale', 'Numéro SIRET', 'Téléphone', 'Adresse', 'E-mail', and 'Date début contrat'. The table contains one row of data. The sidebar on the left includes options for 'DASHBOARD', 'PORTEFEUILLE', 'CLIENTS', 'SITES', 'OBJECTIVES', 'GESTION DES UTILISATEURS', and 'LOGS'. The 'CLIENTS' option is currently selected. The top right of the interface shows the user's name 'BEN OUEZZOU AHMED' and a profile icon.

Code client	Raison sociale	Numéro SIRET	Téléphone	Adresse	E-mail	Date début contrat
d-4512	raison1	125879	+339999999999	france/paris:4202/testpays	test@user.com	2024-07-25

Figure 27 : Interface de gestion des clients

CHAPITRE 3 : RELEASE 1 : GESTION DES UTILISATEURS, LES ROLES, LES SITES ET LES CLIENTS

La **Figure 28** ci-dessous présente l'interface d'ajout des clients dans « ARYANEGO »

The screenshot shows the 'Ajouter Client' (Add Client) form in the ARYANEGO application. The form is a modal window with the following fields:

- Code client: (Required, with error message 'Code client est requis.')
- Raison sociale:
- Description:
- Actif: ☐
- Email:
- Téléphone: (Prefix: +33)
- Date de début contrat: (Format: jj/mm/aaaa)
- Adresse:
- Code postal:
- Ville:

Buttons: AJOUTER, ANNULER

Figure 28 : Interface d'ajout d'un client

La **Figure 29** ci-dessous présente l'interface d'ajout d'un compte client à partir des informations ajoutées précédemment dans « ARYANEGO »

The screenshot shows the 'Edit vos informations personnelles de Client raison1' (Edit personal information of Client raison1) form in the ARYANEGO application. The form is a modal window with the following fields:

- Nom: (Required, with error message 'Merci de renseigner votre Nom.')
- Prénom: (Required, with error message 'Merci de renseigner votre Prénom.')
- E-mail: (Required, with error message 'Veuillez entrer votre adresse e-mail.')
- Mot de passe: (Required)
- Date d'entrée: (Format: jj/mm/aaaa)
- actif: ☐
- Rôle: (Dropdown menu)

Buttons: AJOUTER, CANCEL

Figure 29 : Interface d'ajout un compte client

Conclusion

Dans ce chapitre, nous avons présenté le premier release. Ce premier jalon représente une étape importante dans notre projet, en permettant la gestion des utilisateurs, les rôles, les sites et les clients. À ce stade, l'administrateur peut collaborer avec le négociateur pour gérer les sites, ainsi que superviser la gestion des clients et des utilisateurs autorisés à s'authentifier. Toutes les fonctionnalités planifiées pour ce release ont été réalisées. Dans le chapitre suivant, nous entamons la deuxième release.

CHAPITRE 4

RELEASE 2 : GESTION DES OBJECTIVES, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

Introduction

Ce chapitre présente le deuxième release du projet "Gestion des objectifs, manipulation des portefeuilles, suivre les logs et implémenter le tableau de bord". Ce release met l'accent sur la gestion des objectifs des négociateur et la manipulation des portefeuilles. L'objectif est d'enregistrer tous les primes des négociateurs et de suivre tous les processus de vente ou de location des pylônes. Ainsi que l'administrateur peut suivre les logs des utilisateurs et de prendre une vue d'ensemble des mises à jour grâce aux statistiques du tableau de bord.

1 Backlog du release 2

Le backlog de sprint est composé des éléments sélectionnés à partir du backlog de produit. Ces éléments sont généralement convenus lors de la planification de sprint et représentent le résultat principal de cette dernière. Les **Tableaux 9, 10 et 11** exposeront les tâches qui ont été effectuées au cours du release 2.

1.1 Backlog du sprint 4

ID User story	User Story	ID tâche	Tâches	Priorité
4.1	En tant qu'administrateur, je souhaite créer des objectifs.	1.1	- Préparer l'interface d'ajout d'un objectif. - Implémenter la logique coté serveur pour la création d'un objectif. - Consommer l'api d'ajout d'un objectif	Haute
		1.2		
		1.3		
4.2	En tant qu'administrateur, je souhaite consulter la liste des objectifs.	2.1	- Préparer l'interface de tableau des objectifs. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		2.2		
		2.3		
4.3	En tant qu'administrateur, je souhaite suivre les primes des négociateurs.	3.1	- Préparer l'interface de suivre les primes. - Implémenter la logique coté serveur. - Consommer l'apis de suivre les primes.	Haute
		3.2		
		3.3		
4.4	En tant qu'administrateur, je souhaite filtrer les objectifs.	4.1	- Préparer la section de filtrage et recherche des objectifs. - Consommer l'api de filtrage des objectifs selon les négociateurs et une date spécifique	Haute
		4.2		

Tableau 9 : Backlog du sprint 4

1.2 Backlog du sprint 5

ID	User Story	ID tâche	Tâches	Priorité
5.1	En tant qu'administrateur, je souhaite modifier l'état de prestation de site.	1.1	- Implémenter la logique coté serveur pour la modification de l'état d'une prestation. - Consommer l'api de modification de l'état de prestation.	Haute
		1.2		
5.2	En tant qu'administrateur, je souhaite modifier les prestations des sites.	2.1	- Préparer l'interface de modification d'une prestation de site. - Implémenter la logique coté serveur pour la modification d'une prestation. - Consommer l'api de modification d'une prestation	Haute
		2.2		
		2.3		
5.3	En tant qu'administrateur, je souhaite ajouter des visites.	3.1	- Préparer l'interface d'ajout d'une visite. - Implémenter la logique coté serveur pour la création d'une visite. - Consommer l'api d'ajout d'une visite	Haute
		3.2		
		3.3		
5.4	En tant qu'administrateur, je souhaite consulter à la liste des visites.	4.1	- Préparer l'interface de tableau des visites. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		4.2		
		4.3		

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

5.5	En tant qu'administrateur, je souhaite consulter à la liste d'historisation des appels.	5.1	- Préparer l'interface de tableau d'historisation des appels. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		5.2		
		5.3		
5.6	En tant qu'administrateur, je souhaite chercher des portefeuilles.	6.1	- Préparer la section de filtrage et recherche des portefeuilles. - Consommer l'api de filtrage des portefeuilles selon des critères	Haute
		6.2		
5.7	En tant qu'administrateur, je souhaite exporter les portefeuilles.	7.1	- Préparer la Template de contrat. - Implémenter la logique coté serveur pour la récupération de toutes les données dans le Template. - Consommer l'api d'exportation des portefeuilles.	
		7.2		
		7.3		
5.8	En tant qu'administrateur, je souhaite consulter à la liste des portefeuilles.	8.1	- Préparer l'interface de tableau des portefeuilles. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	
		8.2		
		8.3		

Tableau 10 : Backlog du Sprint 5

1.3 Backlog du sprint 6

ID	User Story	ID tâche	Tâches	Priorité
6.1	En tant qu'administrateur, je souhaite Consulter liste des logs.	1.1	Préparer l'interface des logs.	Haute
		1.2	Consommer les apis de récupération implémentée coté Backend.	
6.2	En tant qu'administrateur, je souhaite exporter les logs dans un fichier Excel.	2.1	Préparer la section d'exportation des logs.	Haute
		2.2	Consommer l'api backend de l'exportation des logs.	
6.3	En tant qu'administrateur, je souhaite consulter au statistique de performance.	3.1	Préparer la section de statistique de performance.	Haute
		3.2	Consommer l'api backend de statistique de performance.	
6.4	En tant qu'administrateur, je souhaite suivre les changements d'état du site.	4.1	Préparer la section de de suivi d'état des sites.	Haute
		4.2	Consommer l'api backend de suivi de changement d'état.	
6.5	En tant qu'administrateur, je souhaite filtrer les sites selon les priorités.	5.1	Préparer la section de filtrages des sites.	Haute
		5.2	Consommer l'api backend de de filtrage des sites.	
6.6	En tant qu'administrateur, je souhaite consulter l'historique d'appels des négociateurs.	6.1	Préparer l'interface de tableau d'historisation des appels.	Haute
		6.2	Implémenter la logique coté serveur pour la récupération de toutes les données.	
		6.3	Consommer l'api de récupération.	

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

6.7	- En tant qu'administrateur, je souhaite consulter aux emplacements des sites.	7.1	- Préparer l'interface de l'emplacement des sites. - Implémenter la logique coté serveur pour la récupération de toutes les données. - Consommer l'api de récupération.	Haute
		7.2		
		7.3		
6.8	En tant qu'administrateur, je souhaite filtrer les contrats des négociateurs.	8.1	- Préparer la section de filtres des contrats. - Consommer l'api backend de de filtrage des contrats.	Haute
		8.2		
6.9	En tant qu'administrateur, je souhaite filtrer la production par prestation et par négociateurs.	9.1	- Préparer la section de filtres des prestations. - Consommer l'api backend de de filtrage des prestations.	Haute
		9.2		

Tableau 11 : Backlog du Sprint 6

2 Raffinement des diagrammes du Release 2

Dans cette section, nous allons nous concentrer sur les diagrammes raffinés de ce Release.

2.1 Diagramme de cas d'utilisation « Gestion des objectifs »

Le diagramme de cas d'utilisation détaillé de la partie Gestion de Livres est représenté dans la **Figure 30**.

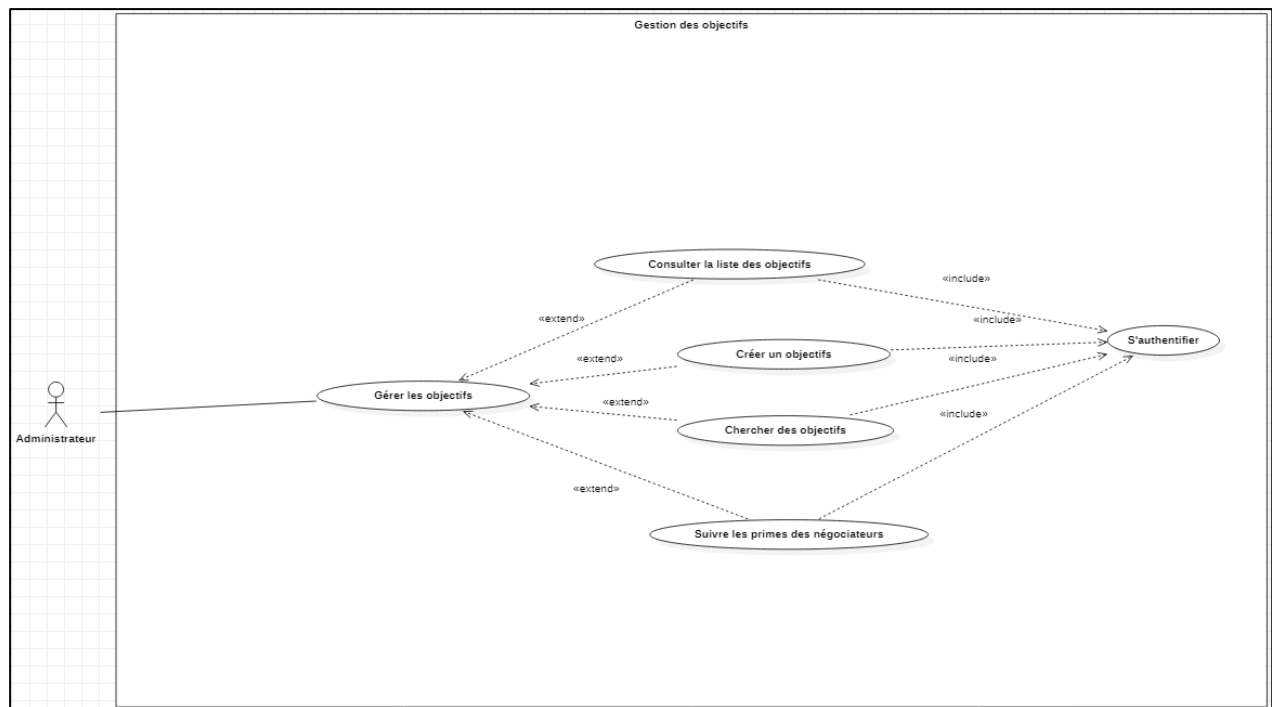


Figure 30 : Diagramme de cas d'utilisation « Gérer des objectifs »

2.2 Diagramme de cas d'utilisation « Manipulation des portefeuilles »

Le diagramme de cas d'utilisation détaillé de la partie Gestion des Secteurs est représenté dans la **Figure 31**.

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS,
MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE
TABLEAU DE BORD

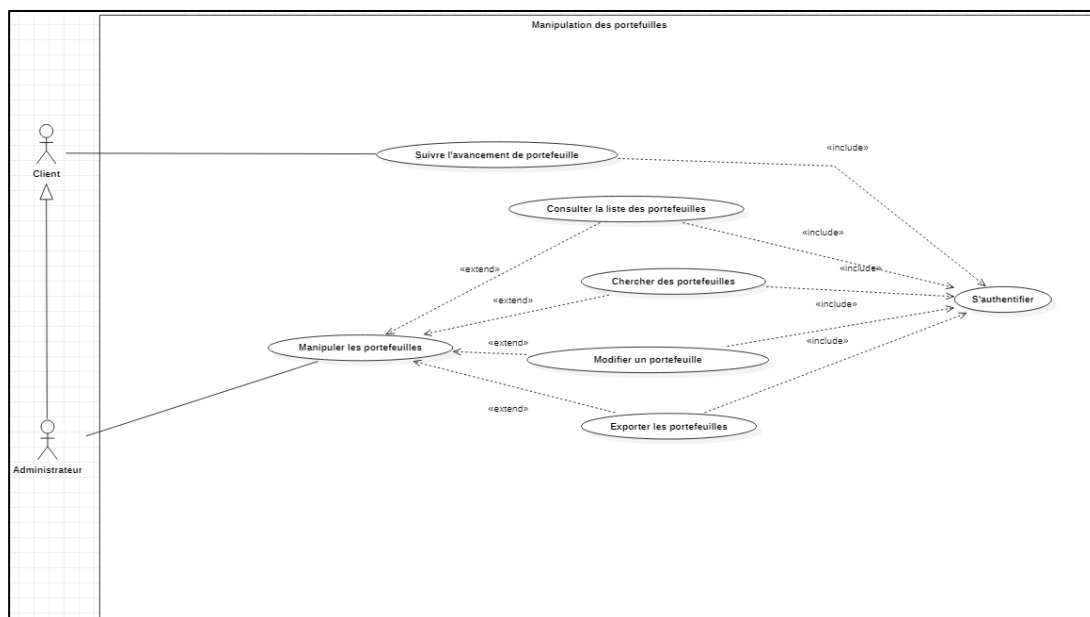


Figure 31 : Diagramme de cas d'utilisation « Manipulation des portefeuilles »

2.3 Diagramme de cas d'utilisation « Suivre les logs »

Le diagramme de cas d'utilisation détaillé de la partie Gestion des Superviseurs est représenté dans la **Figure 32**.

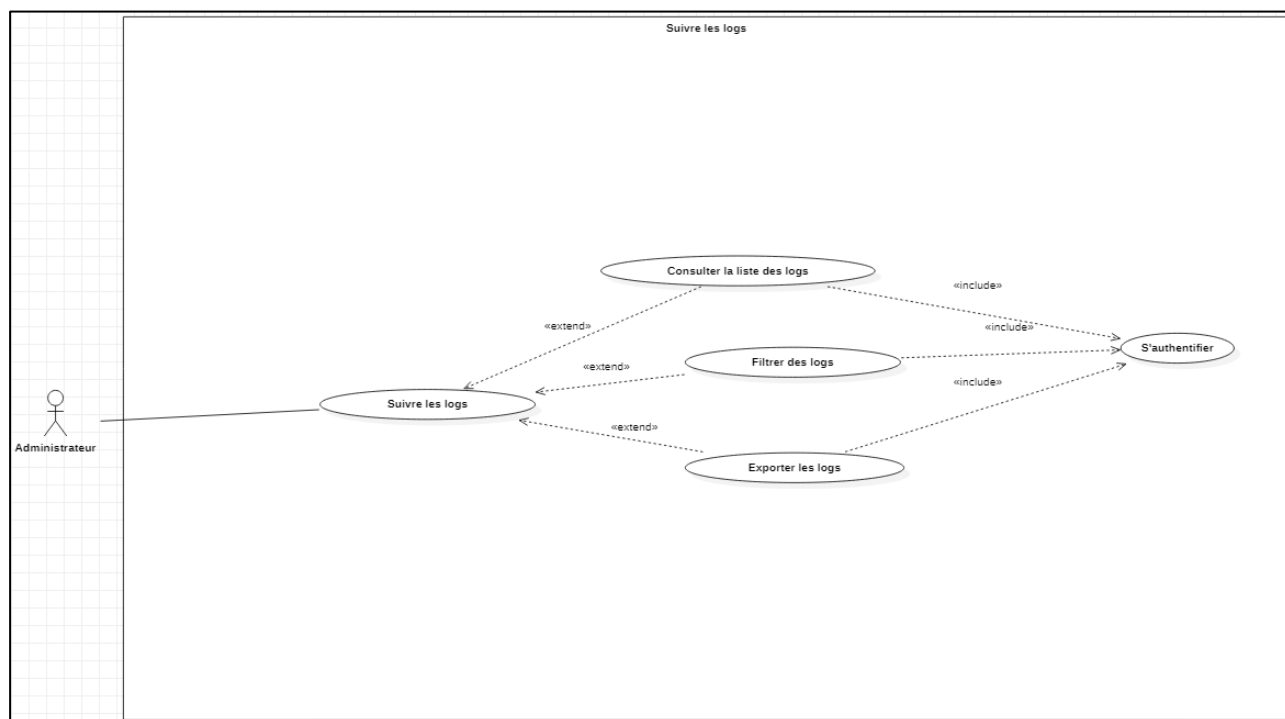


Figure 32 : Diagramme de cas d'utilisation « Suivre les logs »

2.4 Description d'un cas d'utilisation du Release 2

2.4.1 Description du cas « Modifier portefeuille »

2.4.1.1 Description textuelle

Dans cette section, nous présentons la description et le diagramme de séquence du cas d'utilisation « Modifier portefeuille ». La description textuelle correspondante est présentée dans le **Tableau 12** ci-dessous.

Acteur	Administrateur
Pré condition	L'administrateur doit être déjà connecté.
Post condition	Portefeuilles Affichés
Scénario	1. L'administrateur choisit le portefeuille qu'il souhaite modifier et clique sur le nom de site de ce dernier, ce qui le redirigera vers l'interface de modification. 2. Affichage d'un formulaire de mise à jour. 3. Modification des champs. 4. Validation des mises à jour avec le passage au prestation suivante.
Scénario alternatif	1. Affichage d'un message d'erreur si le formulaire n'est pas correctement rempli.

Tableau 12 : Description textuelle du cas « Modifier portefeuille »

2.4.1.2 Diagramme de séquence du cas « Modifier portefeuille »

La **Figure 33** présentée ci-dessous montre le processus de modification d'un portefeuille ainsi que son déroulement.

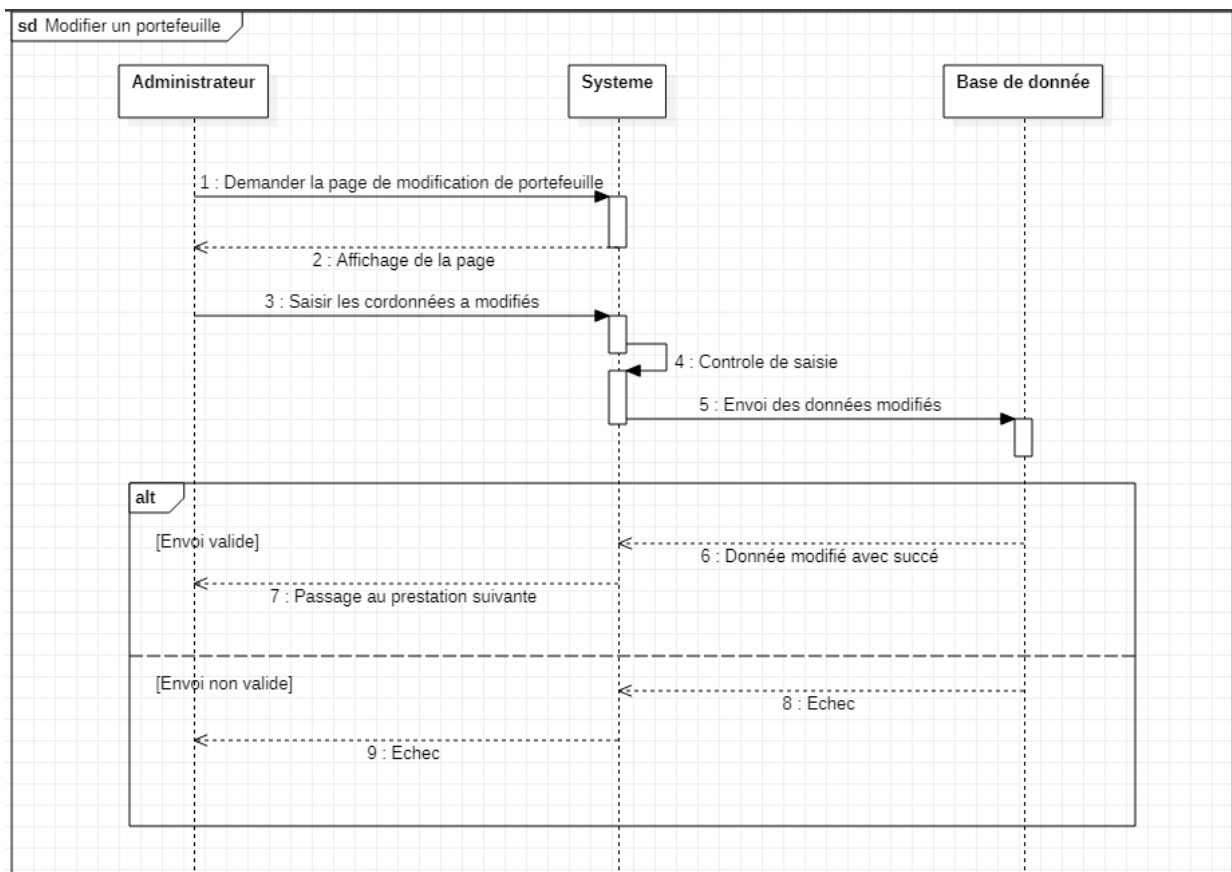


Figure 33 : Diagramme de séquence du cas "Modifier portefeuille"

3 Conception

Le diagramme de classe nous permet d'appréhender la structure globale du système en identifiant les classes principales et leurs relations, sans entrer dans les détails spécifiques de chaque classe ou de leurs attributs. Il offre ainsi une vue d'ensemble de la conception de notre application.

3.1 Etablir Diagramme de classe du release 2

La **Figure 34** présente le diagramme de classe du release 2.

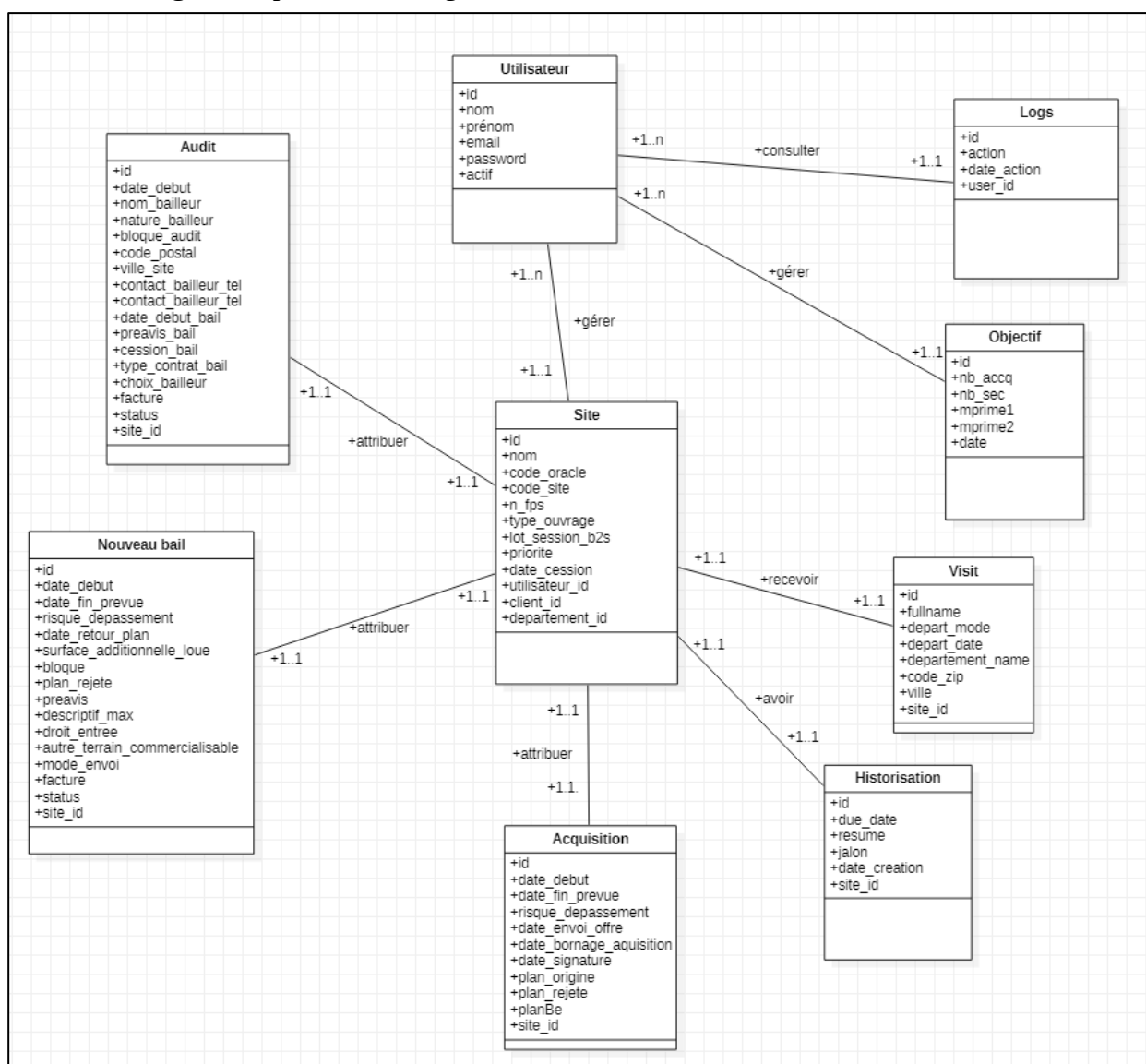


Figure 34 : Diagramme de classe du release 2

4 Réalisation

Dans cette section, nous présenterons quelques interfaces des parties traitées lors de ce release. Ces interfaces permettront d'illustrer concrètement les avancées réalisées dans le développement de notre application.

4.1 Gestion des objectifs

La **Figure 35** ci-dessous présente l'interface de gestion des objectifs dans « ARYANEGO ».

Date	Nombre D'acquisitions	Nombre Sécurisations	Montant Prime (+1)	Montant Prime (2+)
07-2024	3	2	€1,200.00	€212.00
07-2024	2	1	€100.00	€100.00

Figure 35 : Interface de gestion des objectifs

La **Figure 36** ci-dessous présente l'interface de gestion des objectifs dans « ARYANEGO ».

Ajouter Objectif

Date:

Nombre D'acquisitions:

Nombre Sécurisations:

Montant Prime (+1):

Montant Prime (2+):

Négociateur:

Figure 36 : Interface d'ajout d'un objectif

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

La **Figure 37** ci-dessous présente l'interface de gestion des objectifs dans « ARYANEGO ».

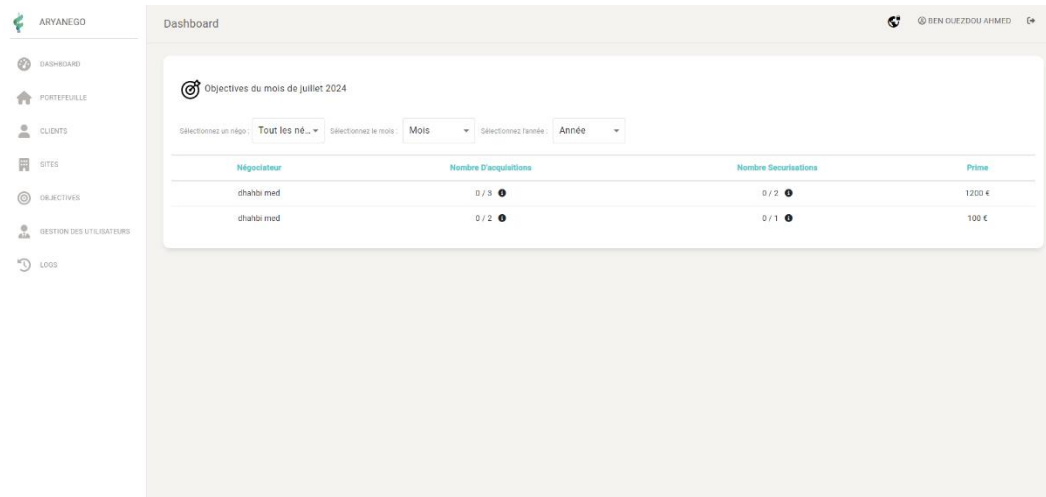


Figure 37 : Interface de détails des objectifs

4.2 Manipulation des portefeuilles

La **Figure 38** ci-dessous présente l'interface de détails des portefeuilles dans « ARYANEGO ».

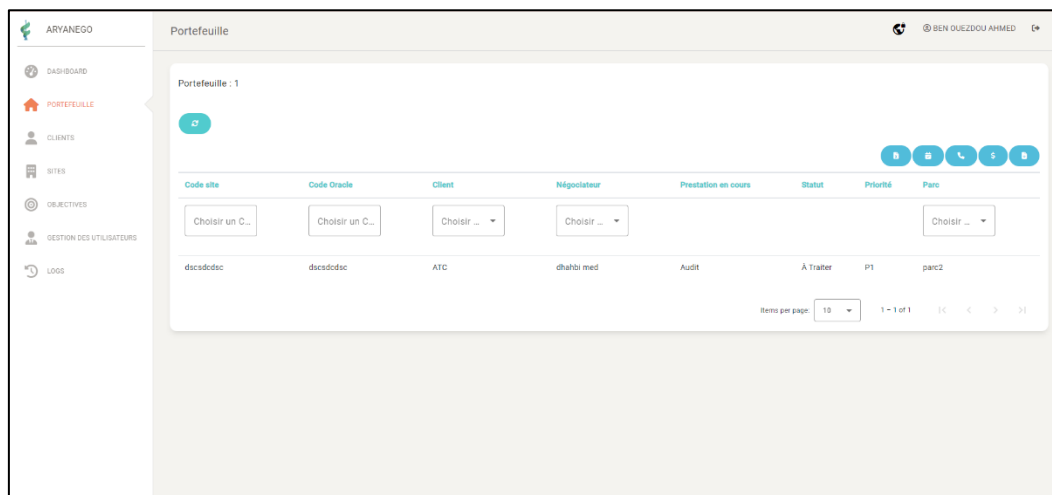


Figure 38 : Interface de détails des portefeuilles

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

La **Figure 39** ci-dessous présente l'interface de modification des portefeuilles dans « ARYANEGO »

Figure 39 : Interface de modification des portefeuilles

La **Figure 40** ci-dessous présente l'interface de gestion d'historique d'appels dans « ARYANEGO »

Criticité	Jalon	Date	Heure	Résumé
Mineur	Attente signature client	Jul 19, 2024	13:35	test2 resume
Critique	En négociation	Jul 26, 2024	13:31	test resume

Figure 40 : Interface de gestion d'historique d'appels

CHAPITRE 4 : RELEASE 2 : CHAPITRE 4 : RELEASE 2 : GESTION DES OBJECTIFS, MANIPULATION DES PORTEFEUILLES, SUIVRE LES LOGS ET IMPLEMENTER LE TABLEAU DE BORD

4.3 Suivre les logs

La **Figure 41** ci-dessous présente l’interface de détails des portefeuilles dans « ARYANEGO »

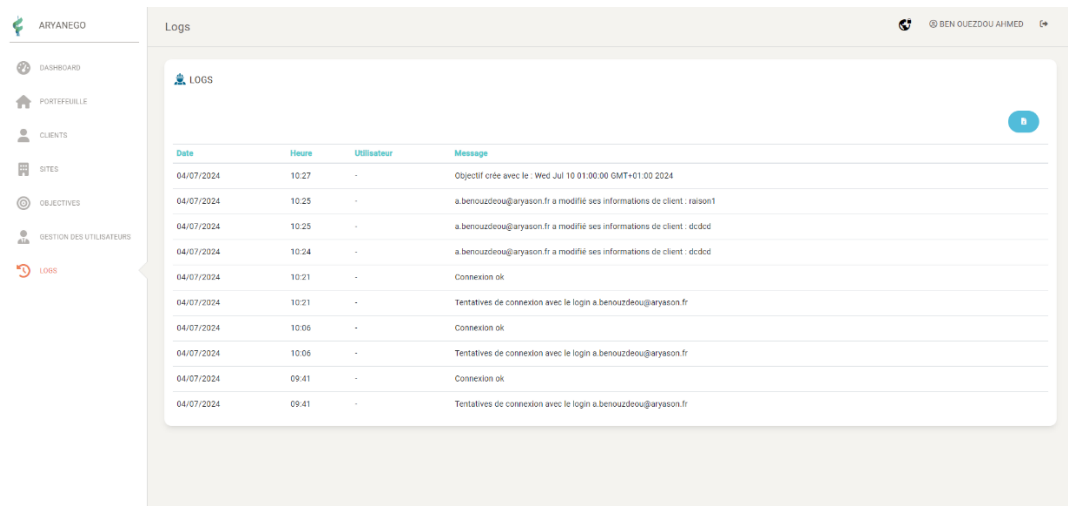


Figure 41 : Interface des logs

4.4 Tableau de bord

La **Figure 42** ci-dessous présente l’interface de détails des portefeuilles dans « ARYANEGO »

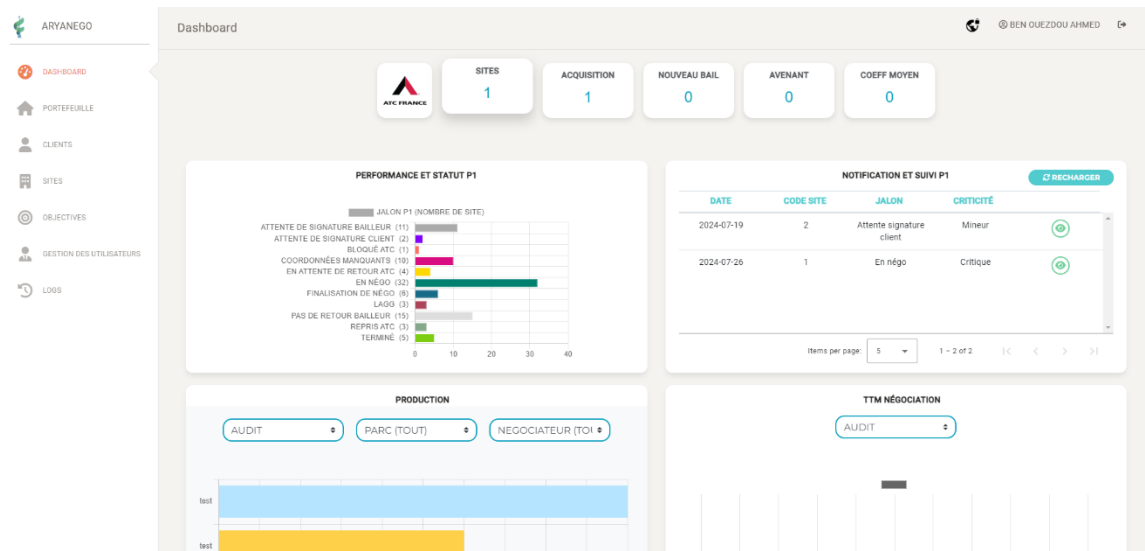


Figure 42 : Interface de tableau de bord

Conclusion

En conclusion de ce deuxième release de notre projet, d'importantes fonctionnalités ont été réalisées dans la gestion des objectifs, manipulation des portefeuilles, suivre les logs, ainsi que l'implémentation de tableau de bord. Ces fonctionnalités visent à renforcer l'efficacité opérationnelle, optimiser les processus de vente et location des sites et améliorer la gestion financière et logistique. Le prochain release se concentrera sur l'intégration de l'automatisation qu'elle donne aux développeurs plus de temps pour se concentrer sur des nouvelles fonctionnalités.

CHAPITRE 5

RELEASE 3 : MISE EN PLACE L'AUTOMATISATION

Introduction

Ce chapitre présente le troisième et dernier release du projet intitulé "Mise en place l'automatisation". Ce release est composé d'un seul sprint, il a pour but de présenter dans son ensemble, l'intégration et la livraison continue de notre projet.

1. Quelque approche

1.1 Approche de l'intégration continue (CI)

Cette approche représente une solution aux problèmes posés par l'intégration de nouveaux segments de code pour les équipes de développement et d'exploitation. Les développeurs pratiquant l'intégration continue fusionnent leurs modifications dans la branche principale aussi souvent que possible. Les modifications du développeur sont validées en créant une version et en exécutant des tests automatisés sur la version. Ce faisant, on évite l'enfer de l'intégration qui survient généralement lorsque les utilisateurs attendent le jour de publication pour fusionner leurs modifications dans la branche de publication. L'intégration continue insiste beaucoup sur l'automatisation des tests pour vérifier que l'application n'est pas endommagée à chaque fois de nouveaux commits sont intégrés dans la branche principale.

1.2 Approche de DevOps

DevOps est un ensemble de pratique qui automatisent les processus entre les deux équipes : Celle de développement et celle des administrations système afin de leur permettre de développer, tester et livrer des logiciels plus rapidement et avec plus fiabilité.

Le concept de DevOps repose sur la mise en place d'une culture de collaboration entre des équipes qui étaient, historiquement, cloisonnées. L'accent est mis sur le changement de mentalité. La collaboration accrue et une intégration plus poussée.

DevOps associe la méthodologie Agile, l'automatisation, la livraison continu et bien plus encore pour aider les équipes de développement et l'équipe d'administration à gagner en efficacité, à innover plus rapidement et à offrir plus de valeur ajoutée aux business et aux clients.

La figure ci-dessous représente le cycle de DevOps

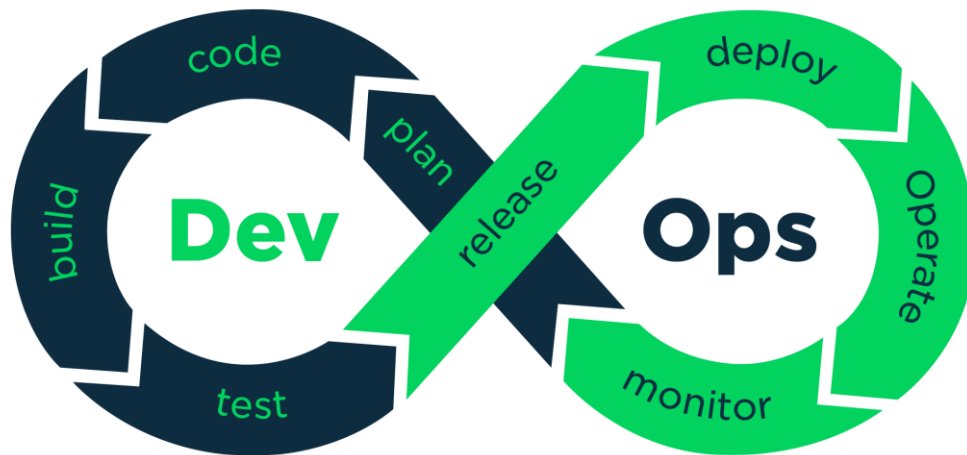


Figure 43 : Cycle de DevOps

2 Pipeline CI : Etude de cas pratique

2.1 Flux CI

Le flux CI décrit le processus automatisé de développement logiciel, depuis l'intégration du code jusqu'à la génération d'une image docker en passant par les tests et la qualité du code. Voici une explication détaillée du flux typique du CI

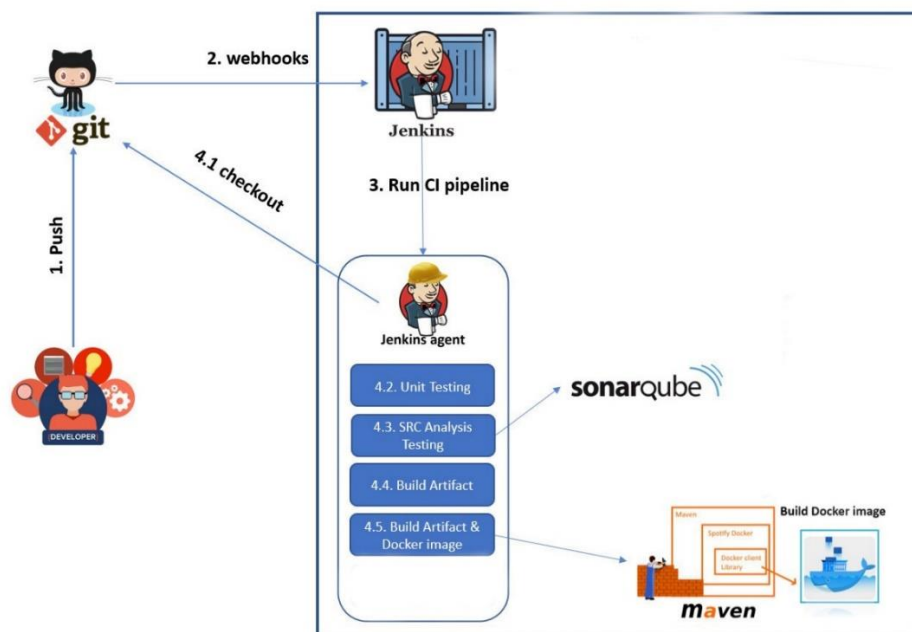


Figure 44 : Les étapes de notre pipeline

2.2 Structure de pipeline

Dans cette section, nous décrivons en détail notre pipeline CI mis en place pour notre application web. Le pipeline comprend les étapes de récupération des modifications, de build, de test et de surveillance.

2.2.1 Etape récupération des modifications

L'objectif de cette étape est de récupérer le code source depuis un dépôt Git pour garantir que la version la plus récente du code est utilisée. Cette étape utilise le plugin Git et elle est déclenchée par chaque push or pull request sur la branche principale.

La figure ci-dessous représente le stage de cette étape

```
stage('GIT') {
  steps {
    echo "Getting Project from Git";
    withCredentials([string(credentialsId: 'git-access-token', variable: 'GIT_TOKEN')]) {
      git branch: "master",
      url: "https://${GIT_TOKEN}@github.com/meddhahbii/nego-app.git"
    }
  }
}
```

Figure 45 : Etape de récupération des modifications

2.2.2 Etape d'exécution des tests unitaires

Cette étape vérifie que chaque unité de code fonctionne correctement et respecter les exigences fonctionnelles et non fonctionnelles, ils sont exécutés à l'aide de Maven pour s'assurer que les modifications n'ont pas introduit des erreurs.

On a utilisé JUnit et Mockito pour faire cette étape

```
stage('Run Unit Tests') {
  steps {
    sh "mvn clean test -Ptest"
  }
}
```

Figure 46 : Etape des tests unitaires

2.2.3 Etape build

L'objectif de l'étape de build est de compiler le code source et de préparer un artefact déployable. Elle utilise Maven pour nettoyer, compiler et emballer notre application et inclut également l'installation de dépendances nécessaires.

```

stage("Build Package") {
    steps {
        sh "chmod +x mvnw"
        sh "mvn clean package -Pprod"
    }
}

```

Figure 47 : Etape de génération de package avec Maven

2.2.4 Etape de construction d'une image docker

L'objectif est de créer une image Docker de notre application pour faciliter le déploiement et l'exécution dans des environnements de conteneurs.

```

stage("Build Docker Image") {
    steps {
        sh "docker build --no-cache -t nego-app ."
    }
}

```

Figure 48 : Etape de génération d'une image Docker

2.2.5 Etape d'analyse SonarQube

L'objectif est d'analyser la qualité de code, détecter les erreurs, les vulnérabilités et les problèmes de maintenabilité. Elle utilise SonarQube pour scanner le code source et fournir un rapport détaillé sur la qualité de code.

```

stage('SonarQube Analysis') {
    steps {
        script {
            sh "chmod +x ./mvnw"
            sh "mvnw sonar:sonar -Dsonar.host.url=${SONARQUBE_SERVER} -Dsonar.login=${SONARQUBE_TOKEN}"
        }
    }
}

```

Figure 49 : Etape de test de qualité de code

3 Réalisation

Dans cette section on va exposer le travail accompli explorer les différentes étapes pour réaliser la partie d'automatisation en détaillant chaque étape.

3.1 Création d'un agent Jenkins

Cette figure présente notre agent sous le nom « Agent-erp-aryatower »

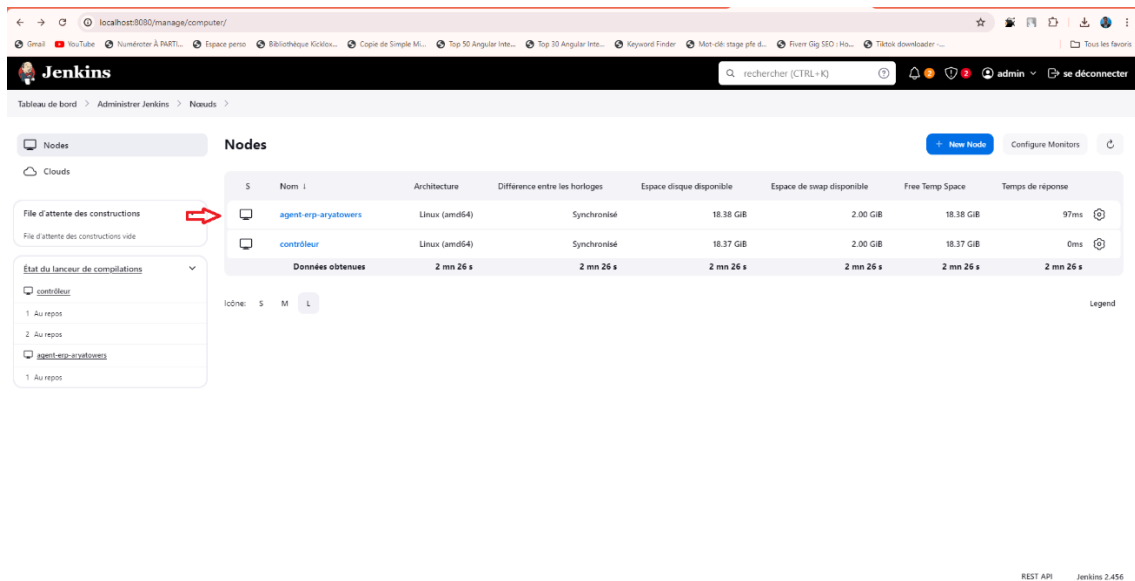


Figure 50 : Interface des agents dans Jenkins

La Figure 51 le pipeline assigné à notre agent

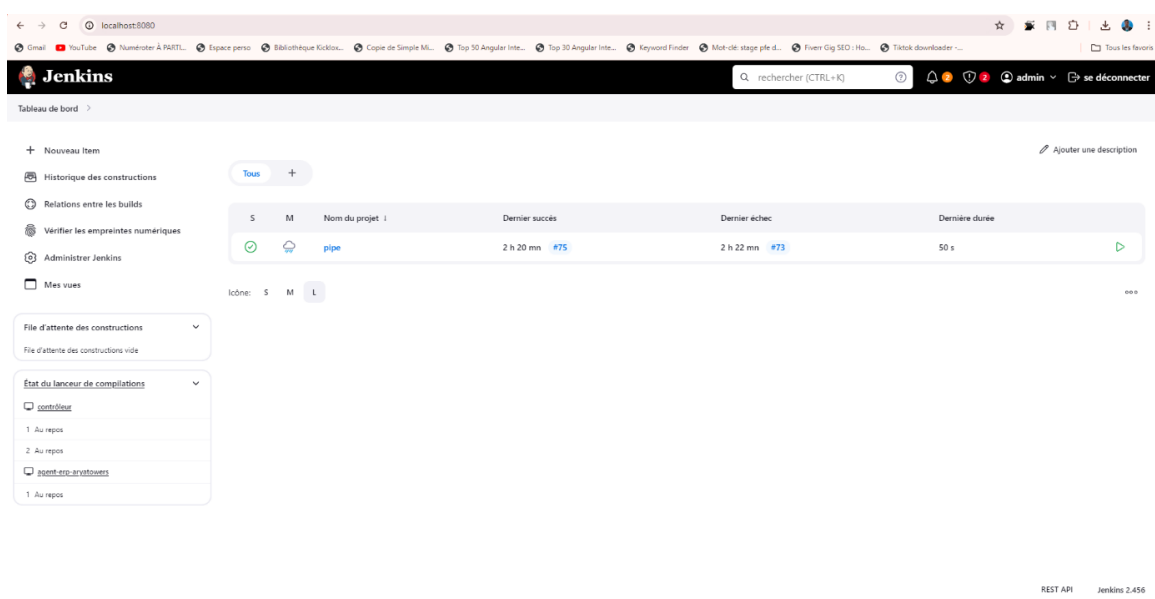


Figure 51 : Pipeline assigné à l'agent « Agent-erp-aryatower »

3.2 Exécution de pipeline

La **Figure 52** présente l'exécution de première étape de notre pipeline qui est la récupération de modification à partir de GitHub.

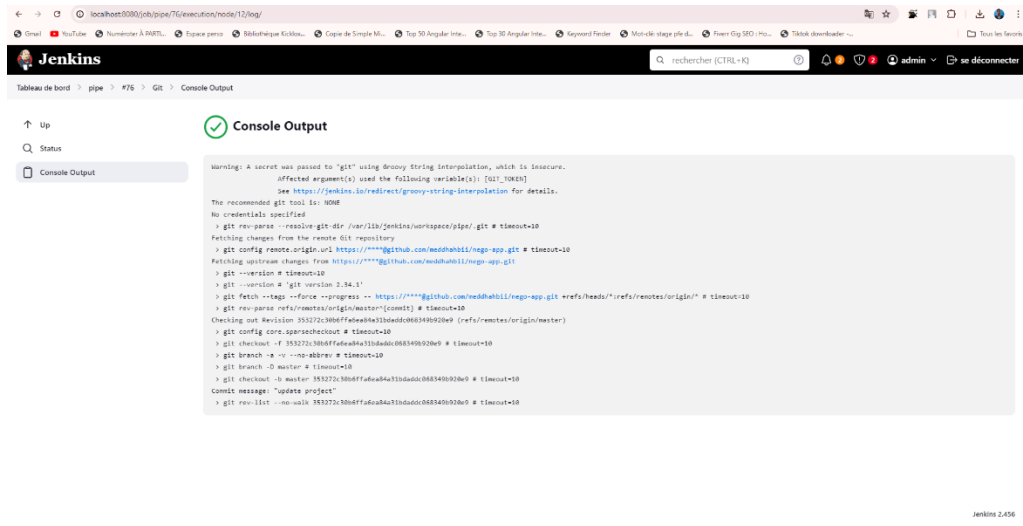


Figure 52 : Exécution de l'étape « GIT »

Passant maintenant par la deuxième étape qui est les tests unitaires et la **Figure 53** montre l'exécution.

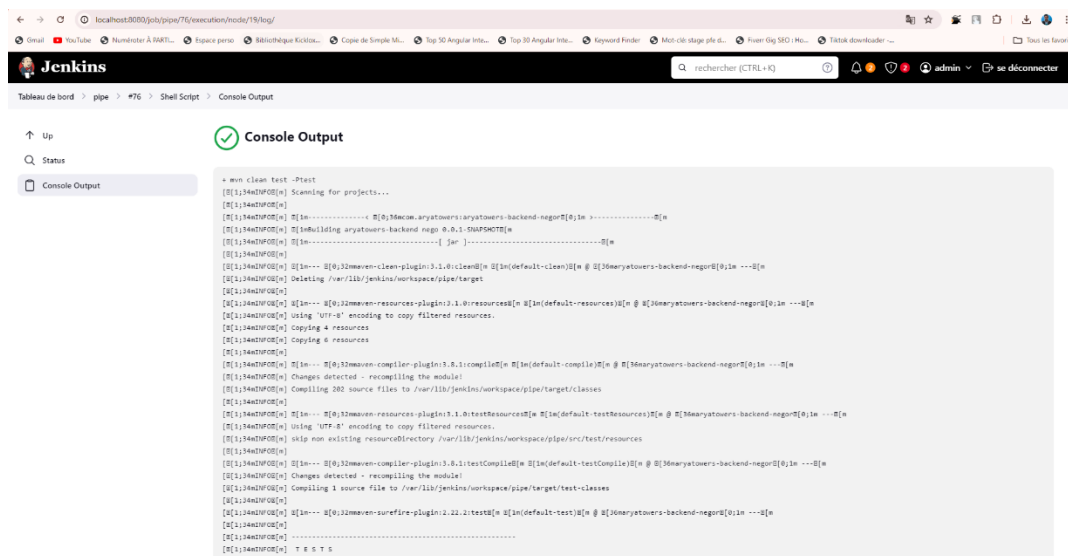


Figure 53 : Exécution de l'étape « Unit tests »

CHAPITRE 5 : RELEASE 3 : MISE EN PLACE L'AUTOMATISATION

La **Figure 54** montre l'exécution de l'étape de construction un package de notre projet

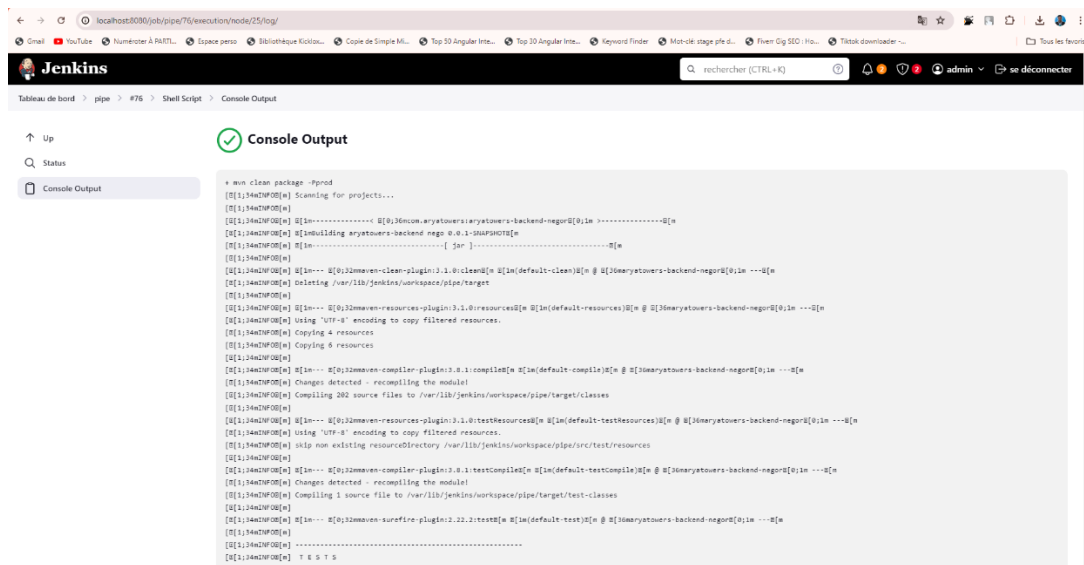


Figure 54 : Exécution de l'étape « Build Package »

La figure ci-dessous présente la génération d'une image docker à partir de notre Dockerfile de notre projet.

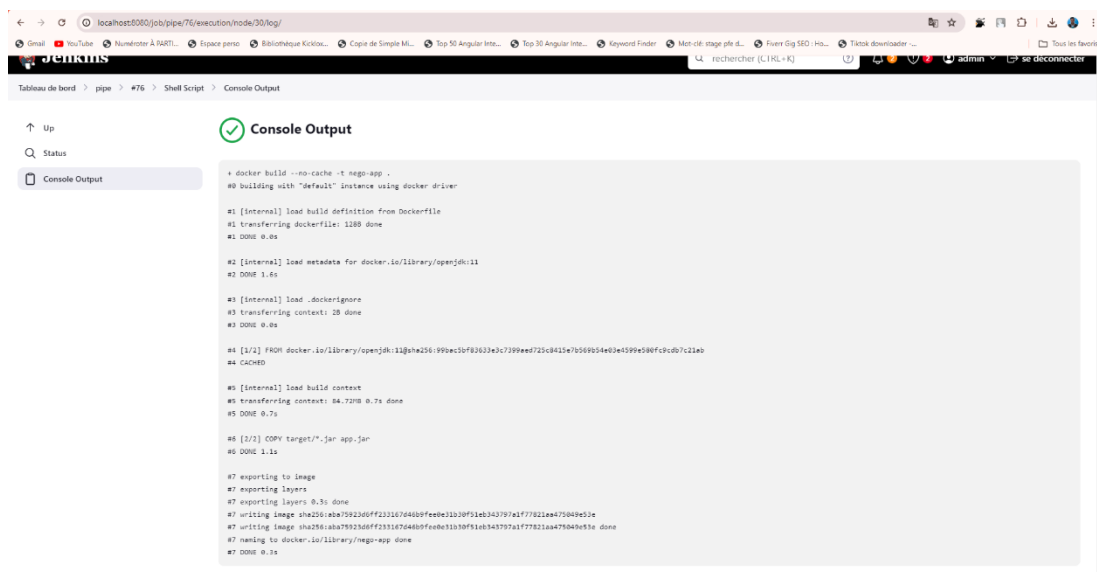


Figure 55 : Exécution de l'étape « Build Docker Image»

Conclusion

À cette étape, nous avons réussi à mettre au point le dernier release de mise en place d'automatisation sur notre projet, nous avons détaillé notre travail de cette partie à l'aide des captures qui présente les différentes étapes de réalisation de cette partie.

CONCLUSION GENERALE

À la fin de ce projet, nous avons réussi à créer l'application « ARYANEGO » sur plusieurs aspects. Cette expérience nous a permis d'appliquer nos connaissances théoriques acquises tout au long de notre cursus, d'approfondir ces connaissances et de découvrir de nouvelles technologies. Nous avons pu développer nos compétences en utilisant Spring Boot et Angular, deux Frameworks puissants pour la création d'applications web. Ce projet nous a également offert l'occasion de renforcer les liens avec les membres de l'entreprise, avec qui nous avons collaboré et coopéré pour mener à bien notre mission et gagner la confiance de notre encadrant et du responsable de l'entreprise. Il est important de souligner que notre travail sur la création du l'« ARYANEGO» ne s'arrête pas là. Nous continuerons à chercher des moyens d'améliorer notre algorithme d'optimisation, en nous basant sur les commentaires et les retours des client. Notre objectif est de fournir des résultats fiables qui répondent aux attentes de nos utilisateurs. Ce projet a été une étape significative dans notre développement professionnel et personnel, et nous sommes impatients de poursuivre notre croissance en contribuant au succès continu de l'entreprise.

WEBOGRAPHIE